

Optimizing LLM Inference: Fluid-Based Online Scheduling under Memory Constraints

(Authors' names are not included for peer review)

Authors are encouraged to submit new papers to INFORMS journals by means of a style file template, which includes the journal title. However, use of a template does not certify that the paper has been accepted for publication in the named journal. INFORMS journal templates are for the exclusive purpose of submitting to an INFORMS journal and are not intended to be a true representation of the article's final published form. Use of this template to distribute papers in print or online or to submit papers to another non-INFORM publication is prohibited.

Abstract. Large Language Models (LLMs) power many modern applications, but their inference procedure poses unique scheduling challenges: the Key-Value (KV) cache grows dynamically during response generation, and memory overflow triggers eviction that can cascade into system-wide failures. Even when memory capacity exceeds the theoretical requirement, conventional scheduling algorithms fail because they do not account for this dynamic memory growth—a system that should be stable can become unstable under poor scheduling.

This paper formulates LLM inference optimization as a multi-stage online scheduling problem. We develop a fluid dynamics approximation to establish a tractable benchmark and derive the Waiting for Accumulated Inference Threshold (WAIT) algorithm. WAIT uses threshold-based batching to prevent eviction by keeping the system near load balance, achieving near-optimal throughput when output lengths are known.

For practical settings where output lengths are unknown at arrival, we introduce Nested WAIT. Rather than predicting output lengths, Nested WAIT classifies prompts on-the-fly: short prompts complete early and exit, while longer prompts naturally advance to later segments. A safety buffer provides high-probability protection against memory overflow with only logarithmic overhead.

Theoretical analysis establishes near-optimal performance in the asymptotic regime. Experiments on Llama-7B with an A100 GPU demonstrate that our approach achieves superior throughput and reduced latency compared to vLLM and Sarathi. This work applies operations research principles to establish a theoretical framework for LLM deployment under memory constraints.

Key words: Large Language Model, Key-value cache, Memory Constraint, Online scheduling

1. Introduction

Large Language Models (LLMs) now dominate natural language processing (NLP) (Devlin et al. 2019, Brown et al. 2020, Kaplan et al. 2020, Ouyang et al. 2022, Wei et al. 2022a,b, Touvron et al.

2023, OpenAI 2024), powering chatbots (Anthropic 2023, Bai et al. 2023, OpenAI 2024, DeepSeek-AI 2025), search engines (Google 2023, Microsoft 2023), and programming assistants (GitHub 2022, Anthropic 2025). These models generate text through *inference*—producing responses to user queries—which imposes significant computational demands on memory and processing resources (García-Martín et al. 2019). Systems like ChatGPT incur daily inference costs exceeding \$700,000 and contribute to substantial carbon emissions (Patel 2023, Patterson et al. 2021). Efficient inference scheduling can reduce these operational costs and energy consumption (Strubell et al. 2019, Desislavov et al. 2021) by balancing system throughput and response latency (Wu et al. 2022).

To design effective scheduling, we must first understand the LLM inference process. Figure 1 provides a step-by-step illustration of how a query is processed during inference. The process begins when a user submits a prompt—e.g., “Is apple a fruit?”—which is then passed through several computational stages. The inference pipeline consists of two main phases:

- **Prefill Phase (stage 0):** Upon receiving the prompt, the model first tokenizes the input into a sequence of discrete units (e.g., “Is”, “apple”, “a”, “fruit”, “?”). These tokens are then embedded and simultaneously processed in a single forward pass to compute the *Key-Value (KV) cache*, which stores intermediate representations (i.e., attention keys and values) for each token. These precomputed values enable efficient reuse during subsequent decoding steps. This phase corresponds to Stage 0 in Figure 1, where all prompt tokens are embedded and their KV representations are added to the cache.

- **Decode Phase (stages 1 to l'):** After the prefill phase, the model enters the decode phase, where it generates the output one token at a time. At each stage, the model queries the existing KV cache to compute the next token, appends the new token to the output sequence, and updates the KV cache with its key-value pair. For instance, the model might first generate “Yes” (Stage 1), then “it” (Stage 2), followed by “is” (Stage 3), and finally the “End of Sequence” (EOS) token (Stage 4). This progression is depicted in the lower part of Figure 1. Each token generation step involves both reading from and writing to the KV cache, resulting in a memory footprint that grows linearly with the length of the generated sequence.

Figure 1 also highlights key performance metrics in LLM inference:

- **Time-To-First-Token (TTFT)** measures the latency from user input to the first generated token.
- **Latency** refers to the total time required to complete the generation of all output tokens.

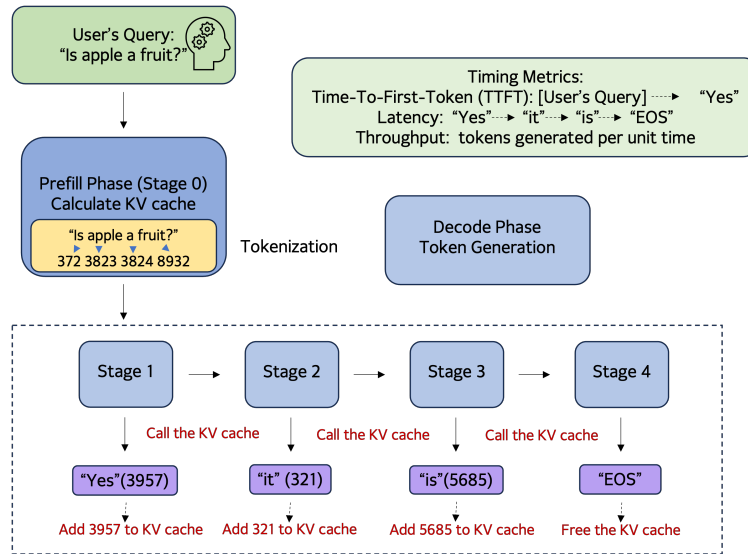


Figure 1 An example of LLM inference.

- **Throughput** captures the average number of tokens generated per unit time.

This inference structure, particularly the growing memory requirements and sequential decoding pattern, introduces fundamental constraints on scheduling and batching. Efficient scheduling must account for both prompt heterogeneity and KV cache dynamics to balance latency and resource use.

Scheduling LLM inference tasks involves grouping prompts into *batches* processed concurrently on a GPU. Batching improves throughput: processing multiple prompts together amortizes the fixed overhead of each iteration and better uses GPU parallelism. However, this benefit comes with a fundamental tension—larger batches consume more memory through their combined KV caches. The KV cache improves efficiency, as it prevents the model from recalculating attention history for each new token; without it, computational cost would scale quadratically with sequence length. But the KV cache also grows dynamically during decode, making memory footprint unpredictable. This creates a core trade-off: *larger batches increase throughput but consume more memory, risking capacity overflow*.

Memory overflow can occur in several practical scenarios: (i) *bursty arrivals*, where sudden traffic spikes temporarily exceed capacity; (ii) *unknown output lengths*, where prompts generate longer responses than anticipated; and (iii) *multi-tenant environments*, where multiple users compete for GPU memory. The unknown output length problem is particularly challenging: since the scheduler

cannot predict how much memory each prompt will eventually consume, it cannot reliably plan batch composition or admission decisions. Traditional scheduling methods such as Shortest Job First (SJF) assume fixed job sizes and known processing times, making them unsuitable when memory demands grow unpredictably (Tay et al. 2022, Kang et al. 2024, Hooper et al. 2025).

The Eviction Challenge. When the total KV cache exceeds GPU memory capacity, the system must evict some in-progress prompts. Two approaches exist: (i) *swapping* to CPU/SSD, incurring I/O overhead (Sheng et al. 2023, Aminabadi et al. 2022); or (ii) *recomputation*, where the KV cache is discarded and the prompt restarts from prefill (Kwon et al. 2023). We focus on recomputation because: (1) it is the default in modern production systems such as vLLM (vLLM Team 2025); and (2) our goal is to *minimize eviction* through intelligent scheduling, rather than optimizing the eviction mechanism itself (Li et al. 2025a). Eviction creates a vicious cycle: freeing memory temporarily, but restarted prompts consume resources again, potentially triggering further evictions—users may experience this as error messages (e.g., “Something went wrong”) or incomplete responses under heavy load.

Memory sufficiency alone does not guarantee stability. The key to maximizing throughput is *load balance*—the equilibrium state where arrival and completion rates match. Yet even when total capacity C exceeds the optimal steady-state memory M^* (derived in our fluid analysis), naive policies can push the system away from equilibrium. For instance, first-come-first-served (FCFS) can trigger cascading evictions that reduce throughput by 12–25

Preventing eviction requires *approaching load balance through controlled admission*. Effective scheduling demands not just sufficient capacity, but maintaining the system near equilibrium by carefully controlling when prompts advance to the next decode stage. This “approaching load balance” principle guides our algorithm design: threshold-based mechanisms hold prompts at decode stage boundaries until memory conditions permit safe continuation, keeping the system close to the balanced fluid dynamics.

Recent system-level optimizations for LLM inference (Yu et al. 2022, Kwon et al. 2023, Agrawal et al. 2023, Pope et al. 2023, DeepSeek-AI 2024, Patel et al. 2024, Zhong et al. 2024) focus on engineering solutions but lack a rigorous mathematical foundation. In practice, output length predictions are imprecise or costly (Fu et al. 2025), and algorithms designed for known output lengths can degenerate significantly when this assumption fails (see Proposition 5).

Our Approach. We develop a fluid dynamics approximation that provides fundamental insights into LLM inference scheduling. The fluid model reveals the equilibrium state—where prompt arrivals and completions balance—and characterizes the optimal steady-state memory M^* that maximizes throughput. This equilibrium serves as a target: when the stochastic system operates near this balanced state, it achieves high throughput while avoiding eviction. The fluid analysis shows that exploiting memory is key to throughput—larger batches process more prompts per iteration, maximizing GPU utilization.

Guided by these fluid insights, we design threshold-based algorithms that keep the stochastic system close to the fluid equilibrium. The WAIT algorithm demonstrates this for known output lengths: threshold-based admission control at each decode stage prevents the system from drifting away from balance. For unknown output lengths, we introduce Nested WAIT, which classifies prompts *on-the-fly*—short prompts complete early and exit, while longer prompts naturally advance to later segments, all without requiring output length prediction. Both algorithms achieve asymptotic optimality by approaching the fluid equilibrium.

1.1. Summary of Contributions

This work contributes to LLM inference scheduling through the following contributions:

- **Memory-Constrained Scheduling Model:** We develop a multi-stage online scheduling model that captures the dynamic growth of KV cache memory during LLM inference, where exceeding capacity triggers costly prompt evictions (Section 2). Using a fluid dynamics approximation from queueing theory, we characterize the optimal throughput and steady-state memory M^* under perfect scheduling (Section 3).
- **Eviction-Prevention Scheduling Algorithms:** We develop threshold-based algorithms that prevent memory overflow and minimize eviction. The WAIT algorithm (Section 4) demonstrates the core insight for known output lengths, while the *Nested WAIT* algorithm (Section 5) extends this to unknown output lengths through on-the-fly type classification. Both achieve asymptotic optimality via coupling arguments.
- **Experimental Validation:** We evaluate our algorithms on synthetic and real-world datasets, outperforming benchmarks like vLLM (Kwon et al. 2023) and Sarathi (Agrawal et al. 2023), which are widely recognized high-performance inference engines, in average throughput using Llama2-7B on a single A100 GPU (Section 6).

These contributions bridge operations research and machine learning, providing a theoretical framework for memory-constrained LLM inference scheduling that explicitly addresses the eviction problem. Memory-constrained LLM inference initially appears as a complex multi-stage stochastic system with feedback through eviction. Our threshold mechanism transforms this into a tractable problem amenable to queueing analysis by decoupling the interactions between prompt types. Our framework is directly applicable to Prefill-Decode Disaggregated systems such as DistServe (Zhong et al. 2024) and Splitwise (Patel et al. 2024), where the decode server handles only decode operations, making the linear iteration time model (Section 2) exact.

1.2. Other Related Work

Online Scheduling Problem and Queueing System. Classical online scheduling problems focus on optimally assigning jobs that arrive sequentially, each with varying completion times. In settings with stochastic arrivals, foundational studies such as Devanur and Hayes (2009), Vee et al. (2010), Cole and Roughgarden (2014), Balkanski et al. (2016), Lattanzi et al. (2020) have developed algorithms that learn arrival patterns or distributions to refine scheduling strategies over time. Beyond individual job processing, works like Im and Moseley (2013), Lucier et al. (2013), Liu and Lu (2015), Li et al. (2020) have explored simultaneous batching and scheduling, grouping similar jobs to enhance efficiency. Within queueing theory, fluid models serve as first-order approximations of stochastic systems, enabling near-optimal control strategies Mandelbaum et al. (1998), Maglaras (2000), Bäuerle (2002), Liu and Whitt (2011), while asymptotic analysis offers insights into long-term system behavior. However, these traditional approaches do not account for key features of LLM inference that complicate asymptotic analysis: (i) iteration time varies with batch composition and current KV cache sizes, precluding fixed service time assumptions; (ii) memory footprint grows dynamically during decode as new tokens are generated; (iii) out-of-memory (OOM) events trigger prompt eviction, creating feedback loops—classical stability and limit theorems assume jobs complete once started, whereas eviction invalidates standard drift arguments.

LLM Inference. Theoretical frameworks for analyzing Large Language Model (LLM) inference are scarce, despite their widespread use. Unlike traditional scheduling, LLM inference involves stochastic arrivals, dynamic memory constraints, and multi-phase processing, complicating analytical efforts. System-level works like Patel (2023) (“Splitwise”) and Zhong et al. (2024) (“DistServe”) split inference into prompt handling and pipeline execution, while scheduling methods such as Yu

et al. (2022), Agrawal et al. (2023), and Agrawal et al. (2024b) use batching to boost throughput—similar to our approach. Other optimizations include DeepSeek-AI (2024) with multi-head latent attention and low-rank KV compression, and Fu et al. (2025) with Kendall’s Tau for prioritizing prompts by predicted output lengths. These works focus on engineering efficiency and system design. From an operations research perspective, several recent works analyze LLM inference scheduling from an online algorithms viewpoint, focusing on competitive ratio and regret bounds. Jaillet et al. (2025) develops an online scheduling algorithm with near-optimal regret guarantees for settings with known output lengths. Wang et al. (2025) studies optimization with variable prefill and decode lengths in pipeline parallelism settings. Chen et al. (2025) addresses robust optimization under prediction uncertainty, considering adversarial scenarios. These works provide worst-case performance guarantees that complement our fluid-based approach, which targets asymptotic optimality in throughput. In parallel, Li et al. (2025b) proposes a stochastic processing model with general batch processing time—including piecewise linear iteration time that captures transitions between compute-bound and memory-bound regimes—demonstrating that work-conserving algorithms achieve optimal throughput. Regarding output length uncertainty, Jaillet et al. (2025) and Wang et al. (2025) assume known output lengths at arrival, Chen et al. (2025) considers adversarial unknown lengths, while our Nested WAIT algorithm addresses stochastic unknown output lengths through on-the-fly classification. Their frameworks employ modeling abstractions tailored to competitive ratio analysis, while our model explicitly captures memory-dependent iteration time (Equation (1)) to analyze the throughput-memory trade-off and eviction prevention mechanisms central to our contribution.

1.3. Notations

For integer $n \geq 1$, we denote $[n] = \{1, 2, \dots, n\}$ as the set of integers from 1 to n . For $x \in \mathbb{R}$, denote $\lceil x \rceil$ as the smallest integer not smaller than x and $\lfloor x \rfloor$ as the largest integer not greater than x . Denote $x_+ = \max\{x, 0\}$. For set S , denote $|S|$ as its cardinality. For two functions $f(T)$ and $g(T)$, we use $f(T) = O(g(T))$ if there exists constant $c_1 > 0$ such that $f(T) \leq c_1 g(T)$ as $T \rightarrow +\infty$ and $f(T) = \Omega(g(T))$ if there exists constant $c_2 > 0$ such that $f(T) \geq c_2 g(T)$ as $T \rightarrow +\infty$.

2. Model

We formalize the online scheduling problem for Large Language Model (LLM) inference under memory constraints. Unlike traditional scheduling where job sizes are fixed, LLM inference involves

dynamically growing memory demands: the Key-Value (KV) cache expands continuously as prompts generate output tokens, and the two-phase structure (prefill and decode) creates fundamentally different resource consumption patterns. This dynamic growth, combined with unknown output lengths, distinguishes our problem from classical queueing models. We first introduce the inference process, then describe prompt characteristics, batching rules, system constraints, and performance metrics.

2.1. Notation

We summarize the key notation used throughout this paper in Table 1.

Table 1 Summary of Notation

Symbol	Description
m	Number of prompt types
j	Index of prompt type, $j \in \{1, \dots, m\}$
λ_j	Arrival rate of type- j prompts (Poisson process)
l_j	Input length (tokens) of type- j prompts after prefill
l'_j	Output length (tokens) of type- j prompts in decode phase
s	Stage index: $s = 0$ for prefill, $s \in \{1, \dots, l'_j\}$ for decode
k	Segment index in Nested WAIT, $k \in \{1, \dots, m\}$
b	Batch index (iteration count)
n_{js}^t	Number of type- j prompts at stage s at time t
n_j	Threshold for type- j prompts in WAIT algorithm
n_k	Threshold for segment k in Nested WAIT
C	GPU memory capacity
$\mathcal{G}^t$	Set of prompts with KV cache on GPU at time t
B^t	Batch of prompts processed at iteration t ($B^t \subseteq \mathcal{G}^t$)
τ	Iteration time to process a batch
d_0	Fixed overhead time per batch iteration
d_1	Time cost per unit of KV cache memory
M^*	Equilibrium memory usage in fluid model
n_j^*	Equilibrium number of active type- j prompts
Throughput*	Equilibrium throughput (fluid benchmark)
Π	Class of admissible scheduling policies

2.2. Prompts and Inference Process

User queries, called *prompts*, arrive at a single Graphics Processing Unit (GPU) for processing. We focus on maximizing how fully the GPU is used, and our algorithms extend to multi-GPU settings such as pipeline parallelism, where each prompt is processed across different GPUs in different layers of the LLM. When communication costs are negligible, our algorithms apply directly to these settings.

During the inference process, each prompt is divided into discrete units known as *tokens* during the prefill phase and generates output text through an inference process in the decode phase. A token represents a distinct text element, such as a word or subword, and its processing relies on a memory structure called the Key-Value (KV) cache. The KV cache improves inference efficiency by storing the computational history of prior tokens, thus avoiding redundant computations. The cache size increases as tokens are processed, requiring careful memory management to maintain system performance.

Prompt Characteristics: Prompts arrive stochastically and are classified into m distinct types according to their input / output lengths, indexed by $j \in \{1, \dots, m\}$. We assume that prompts of type j arrive according to a Poisson process with rate λ_j . Each prompt's memory footprint depends on its processing stage. A prompt of type j has the following properties:

$$\begin{cases} \text{a length of } l_j \text{ tokens after prefill;} \\ \text{a length of } l_j + l'_j \text{ tokens after decode.} \end{cases}$$

Therefore, a type- j prompt must undergo one prefill iteration followed by l'_j decode iterations to complete its inference processing. During the prefill phase, the prompt's input context and question are embedded as l_j tokens in a single iteration, consuming a KV cache of size l_j units. In the subsequent decode phase, one output token is generated per iteration, each incurring an additional unit of KV cache memory.

To track processing progress, we define a prompt to be in **stage** $s = 0$ if it is about to undergo the prefill phase, and in **stage** s (where $s \in \{1, \dots, l'_j\}$) if it is scheduled to receive its s -th output token during the decode phase. Thus, a prompt at stage s occupies memory of size $l_j + s$ units.

Though the total number of different types m can be large (for example, from 1 to 10000), our algorithms and their theoretical guarantees exhibit only a weak (logarithmic) dependence on m . The analysis relies primarily on the total arrival rate across all types, ensuring that our approach scales efficiently regardless of the number of types.

While our analysis focuses on fixed arrival rates λ_j , our threshold-based algorithms can be extended to settings with time-varying arrivals by dynamically adjusting thresholds based on the arrival rates (see Section 7).

2.3. Batching and Iteration Time

To use GPU resources efficiently, the GPU processes tasks in *batches*, combining prompts for prefilling and prefilled prompts for decoding. Figure 2 illustrates the temporal dynamics of batching and scheduling in GPU-based inference. Two prompts, $P1$ and $P2$, arrive sequentially and are processed according to their current stage—prefill or decode—at each time step. Initially, $P1$ arrives and enters the prefill phase, embedding its input sequence “How are you?” in a single iteration. Shortly after, $P2$ arrives with the query “Is 5 a prime?”, which is also scheduled for prefill. The GPU batches prompts in the same phase together; thus, $P1$ is processed for prefill alone, followed by $P2$. At the next step, $P1$ enters the decode phase, generating its first output token “I’m,” while $P2$ undergoes prefill in parallel. Subsequently, both prompts advance in the decode phase, each generating one token per iteration: “Yes” for $P2$ and “I’m fine” for $P1$. This example highlights how the scheduler interleaves prompts in different stages—prefill and decode—into shared GPU batches, thereby improving throughput. Prefill stages involve initializing KV caches based on input tokens, while decode stages extend outputs one token at a time, with incremental KV cache updates.

In general, consider a batch composed of:

- n_1 prompts in the prefill phase (stage $s = 0$), where prompt i has input length $l_{j(i)}$ tokens corresponding to its type $j(i)$. These prompts are processed to initialize their KV caches.
- n_2 prompts in the decode phase, where prompt i is at stage $s_i \in \{1, \dots, l'_{j(i)}\}$, having generated s_i output tokens. The total KV cache size for prompt i is $l_{j(i)} + s_i$ tokens.

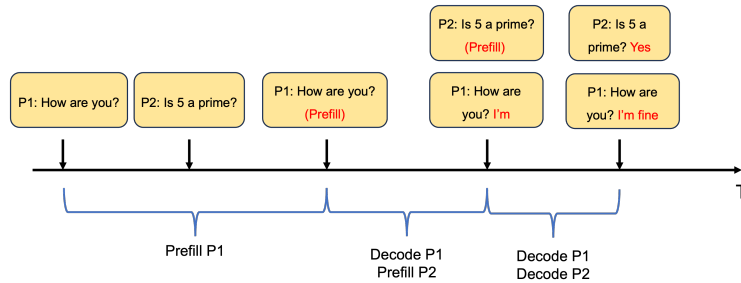


Figure 2 Example of batching and scheduling.

Experiments in [Agrawal et al. \(2023\)](#), [Kwon et al. \(2023\)](#), [Zhong et al. \(2024\)](#) show that *iteration time* scales linearly with total KV cache size. We model this as:

$$\tau = d_0 + d_1 \cdot \underbrace{\left(\sum_{i=1}^{n_1} l_{j(i)} + \sum_{i'=1}^{n_2} (l_{j(i')} + s_{i'}) \right)}_{\text{Total KV cache size in batch}}, \quad (1)$$

where d_0 denotes the fixed overhead time associated with processing a batch, and d_1 represents the time cost per unit of KV cache memory. Equation (1) shows that iteration time scales linearly with total KV cache size. The fixed overhead d_0 has an important implication: *to maximize throughput, we should use memory as fully as possible*. Larger batches amortize the fixed cost d_0 across more prompts, improving efficiency. However, this must be balanced against the memory constraint (2) to avoid eviction. Figure 3 provides a numerical validation of this modeling. We deploy Llama-7B and Llama-13B on a single L20 GPU to test time cost per iteration, using batches consisting of different number of tokens repeated 20 times. The result is highly consistent with our model.

The linear model (1) is most accurate when: (i) batches are decode-dominated, as each decode iteration processes one token per prompt and the computational cost is dominated by KV cache memory access; (ii) the system employs prefill-decode disaggregation (Zhong et al. 2024, Patel et al. 2024), where the decode server handles only decode operations, making Eq. (1) exact. In mixed prefill-decode batches, the linear model provides a good approximation when prefill lengths are similar across prompts.

More generally, iteration time can exhibit piecewise linear behavior across different operating regimes. For example, Li et al. (2025b) models iteration time as $\tau(b') = c + a \cdot \max\{0, b' - b_0\}$, where iteration time is constant for small batch sizes and grows linearly beyond a threshold b_0 . This captures the transition between compute-bound and memory-bound regimes: when batches are small, GPU cores are the bottleneck; when batches are large, memory bandwidth dominates. Our algorithms and analysis extend naturally to such piecewise linear models. The threshold mechanism in WAIT (Section 4) and Nested WAIT (Section 5) controls the number of prompts in each decode stage, ensuring the system operates within safe memory limits regardless of whether iteration time is strictly linear or piecewise linear. The key insight—balancing arrivals and completions to approach the fluid equilibrium—remains valid as long as iteration time scales predictably with batch composition. We adopt the simplified linear model (1) for tractability in our theoretical analysis, while noting that the empirical validation in Figure 3 confirms this approximation is accurate in the decode-dominated regime where our algorithms operate.

2.4. Memory Constraint and Challenges

Unlike traditional scheduling where job sizes are fixed, LLM inference faces a unique challenge: prompts occupy memory that **grows during decode**. Each generated token adds one unit to the

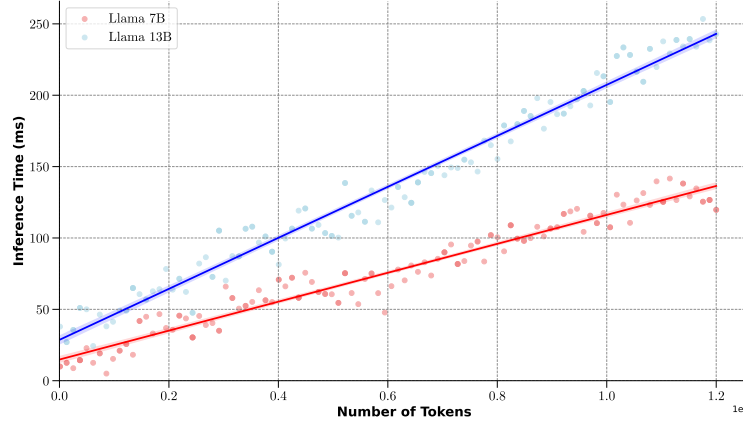


Figure 3 Inference time of Llama-7B and Llama-13B with different number of tokens in the batch on a single L20 GPU

prompt's KV cache, causing the memory footprint to expand continuously until completion. This dynamic growth creates eviction risks even when total capacity appears sufficient.

Memory Constraint: At all times t , the total KV cache stored on GPU must not exceed capacity C :

$$\sum_{i \in \mathcal{G}^t} (l_{j(i)} + s_i^t) \leq C, \quad (2)$$

where $\mathcal{G}^t$ denotes the set of prompts whose KV caches are stored on GPU at time t . This includes both prompts actively processed in the current batch $B^t \subseteq \mathcal{G}^t$ and prompts that are preempted but retain their KV caches on GPU. Here $j(i)$ is the type of prompt i , and $s_i^t \in \{0, 1, \dots, l'_{j(i)}\}$ is its current stage. Unlike traditional scheduling where job sizes are fixed, the memory footprint $l_{j(i)} + s_i^t$ grows dynamically as decode progresses (s_i^t increases each iteration). This dynamic growth creates the risk of *eviction*: if memory becomes insufficient to satisfy the constraint (2), ongoing prompts must be removed from the GPU and their KV caches discarded. When eviction is necessary, prompts are removed in **LIFO order** (most recently admitted prompts are evicted first) until sufficient memory is freed. Evicted prompts lose all accumulated computation and must restart from the prefill phase (stage $s = 0$), re-entering the scheduling queue as new arrivals. This creates a vicious cycle: eviction temporarily frees memory, but restarted prompts compete for resources again, potentially triggering further evictions—a phenomenon we call *cascading failure*. The following example illustrates this mechanism.

EXAMPLE 1 (DYNAMIC MEMORY GROWTH AND EVICTION). Consider a simple prompt type: users submit “Hello” queries, each expecting a 1-token response “Hi.” Each prompt has prefill length $l = 1$ token and decode length $l' = 1$ token. Memory consumption evolves as follows:

- **Stage $s = 0$ (awaiting prefill):** No memory allocated yet.
- **After prefill:** The prompt's KV cache occupies $l = 1$ unit of memory.
- **After decode (stage $s = 1$):** The prompt occupies $l + s = 1 + 1 = 2$ units before completion.

Suppose the GPU has capacity $C = 12$ units. A batch processes 4 prompts at prefill (using $4 \times 1 = 4$ units) and 4 prompts at decode (using $4 \times 2 = 8$ units), totaling $4 + 8 = 12$ units with 4 completions per iteration.

Now consider a scenario where 6 new prompts arrive simultaneously while 3 prompts are already decoding:

1. **Current state:** 3 prompts in decode occupy $3 \times 2 = 6$ units.
2. **Admit all 6 for prefill:** After prefill, these 6 prompts advance to decode. The 3 existing decode prompts complete. System now has 6 prompts in decode, occupying $6 \times 2 = 12$ units—**memory is full**.
3. **Next batch:** 4 new prompts arrive. To prefill them (needing $4 \times 1 = 4$ units), we must **evict** 2 decode prompts to free space.
4. **Evicted prompts restart:** The 2 evicted prompts lose their KV caches and return to stage $s = 0$, re-entering the queue as new arrivals.
5. **Cascade:** These restarted prompts compete with new arrivals. The imbalance persists, triggering repeated evictions.

This example reveals the **cascading eviction problem** our algorithms target. Even with $C \geq M^*$ (the equilibrium memory from our fluid analysis, Section 3), poor admission control triggers this cascade, making the system unstable. The eviction-restart cycle can persist indefinitely, degrading throughput significantly (Example 2 quantifies this for FCFS, showing 12-25% throughput loss).

Core Challenge. When output lengths are unknown, optimistic admission risks overflow; pessimistic admission wastes capacity. Our threshold-based algorithms resolve this dilemma by: (i) preventing evictions through controlled admission, (ii) exploiting memory fully via batching, (iii) classifying prompts on-the-fly as they complete decode stages. This approach maintains the system near the load-balanced equilibrium derived in Section 3, where arrivals match completions and memory use is stable.

2.5. Performance Metrics

We define the metrics guiding scheduling optimization:

- **Throughput**: Average number of tokens generated in decode phase per unit time, measuring system efficiency. We count throughput only after prompt completion to ensure validity of the output.
- **Latency**: Average elapsed time from prompt arrival to completion, capturing responsiveness.
- **Time to First Token (TTFT)**: Average delay from prompt arrival to the generation of the first token, reflecting initial user-perceived delay.

2.6. Optimization Problem and Policy Space

We formalize the problem as optimizing throughput subject to latency and TTFT constraints over continuous time horizon $[0, T]$:

$$\begin{aligned}
 \max_{\pi \in \Pi} \quad & \mathbb{E} [\text{Throughput}^{(T, \pi)}], \\
 \text{s.t.} \quad & \mathbb{E} [\text{Latency}^{(T, \pi)}] \leq L^T, \\
 & \mathbb{E} [\text{TTFT}^{(T, \pi)}] \leq F^T, \\
 & \sum_{i \in \mathcal{G}^t} (l_{j(i)} + s_i^t) \leq C, \quad \forall t.
 \end{aligned} \tag{3}$$

Policy Space Π . We define the class of admissible scheduling policies Π . At each decision epoch t , a policy $\pi \in \Pi$ observes the system state and makes scheduling decisions subject to the following constraints:

Information Structure. The system state at time t includes: (i) the set of all prompts that have arrived by time t , along with their input lengths l_i ; (ii) the current stage s_i^t of each prompt i (i.e., how many output tokens have been generated); (iii) whether each prompt's KV cache is currently stored on GPU. Policies are *non-anticipating*: decisions at time t depend only on information available at or before time t . In particular, the output length l'_i of a prompt may be unknown until the prompt completes (generates an end-of-sequence token). This unknown output length is a key challenge addressed by our Nested WAIT algorithm (Section 5).

Admissible Actions. At each decision epoch, a policy may:

1. **Admit**: Select a subset of waiting prompts (at stage $s = 0$) to enter the system. Admitted prompts wait until included in a batch for prefill.

2. **Batch:** Form a batch B^t of prompts to process in the current iteration. After prefill completes, a prompt's KV cache is allocated on GPU.
3. **Preempt:** Pause a prompt while retaining its KV cache on GPU. The prompt can resume later without recomputation.
4. **Evict:** Remove a prompt's KV cache from GPU to free memory when the constraint (2) cannot be satisfied. Eviction follows a **LIFO (Last In, First Out)** policy: the most recently admitted prompts are evicted first. The evicted prompt returns to the waiting queue at stage $s = 0$, losing all prior computation.

Memory Constraint. At all times, the total KV cache stored on GPU must satisfy the capacity constraint (2).

Preemption and eviction are both permitted, but eviction is costly: the evicted prompt re-enters the system as a new arrival and must repeat all processing stages from the beginning. Our WAIT algorithms are designed to *prevent eviction* by controlling admission through thresholds, thereby avoiding the memory pressure that causes eviction.

3. Fluid Dynamics and Equilibrium

Unlike traditional queueing systems where job sizes are fixed, LLM inference faces a unique challenge: KV cache memory grows dynamically as prompts progress through decode stages. Each additional generated token increases the prompt's memory footprint, making it difficult to predict when total memory usage will exceed capacity C and trigger costly evictions. These evictions waste the partial computation already invested in interrupted prompts, degrading both throughput and latency.

We develop a fluid model to characterize the system's equilibrium state—a balanced configuration where arrival rates match completion rates and memory usage stabilizes. This equilibrium serves two purposes. First, it quantifies the maximum sustainable throughput **Throughput*** under memory constraint C , establishing a benchmark for any scheduling policy. Second, it reveals the load-balancing principle that guides our algorithm design (Section 4): maintain the system close to equilibrium to prevent evictions. When prompts distribute evenly across processing stages, memory consumption remains predictable and overflow risk minimizes.

We first analyze a system with a single prompt type to build intuition about equilibrium dynamics, then extend to multiple heterogeneous types. The equilibrium provides a theoretical benchmark;

our WAIT algorithm (Section 4) achieves asymptotic optimality by approaching this benchmark through threshold-based admission control that keeps the system near the balanced state.

3.1. Single-Type Fluid Model

We begin with a system handling prompts of a single type j . Each prompt's memory usage depends on its processing stage: l_j tokens in the prefill stage ($s = 0$) and $l_j + s$ tokens in the s -th decode stage ($s = 1, \dots, l'_j$). When the prompt completes stage $s = l'_j$, it finishes decoding and its KV cache is cleared from GPU memory.

In equilibrium, we assume the system maintains n_j^* active prompts distributed evenly across all $(l'_j + 1)$ stages, with $n_j^*/(l'_j + 1)$ prompts at each stage. This balanced distribution is crucial: if prompts accumulate at later decode stages—where memory consumption is highest ($l_j + l'_j$ tokens)—total usage may exceed capacity C , forcing evictions that waste partial computation. By spreading prompts evenly, we minimize overflow risk while maximizing throughput. This equilibrium model allows us to derive the system's fundamental capacity limit.

Before deriving the equilibrium parameters, we identify when the system becomes fundamentally unstable—regardless of the scheduling policy used. Intuitively, each prompt requires total processing time $d_1(l'_j + 1)(l_j + l'_j/2)$ to load its KV cache across all $(l'_j + 1)$ stages. When the arrival rate exceeds the inverse of this processing time, the queue grows unbounded.

PROPOSITION 1. *When the condition $\lambda_j d_1(l'_j + 1) \left(l_j + \frac{l'_j}{2}\right) \geq 1$ is satisfied, the system becomes unstable in the sense that the expected average latency under any scheduling policy with deterministic arrival rate λ_j grows linearly with time. Formally,*

$$\mathbb{E}[\mathbf{Latency}^{(T,\pi)}] = \Omega(T) \quad \text{as } T \rightarrow \infty.$$

This condition represents the fundamental capacity limit: arrivals outpace completions, causing latency to grow linearly with time. The proof constructs an equilibrium batch and shows that even in this balanced state, arrivals exceed completions when the condition holds (Appendix A.1). We therefore assume $\lambda_j d_1(l'_j + 1) \left(l_j + \frac{l'_j}{2}\right) < 1$ for the remainder of this section, ensuring the system can reach equilibrium.

The total memory usage in equilibrium depends on the distribution of prompts across stages:

$$M_j^* = \frac{n_j^*}{l'_j + 1} \sum_{s=0}^{l'_j} (l_j + s) = n_j^* \left(l_j + \frac{l'_j}{2} \right), \quad (4)$$

where we sum over all $(l'_j + 1)$ stages, each containing $n_j^*/(l'_j + 1)$ prompts. The factor $(l_j + l'_j/2)$ represents the average memory per prompt across its lifecycle, from prefill (l_j) to final decode stage (l'_j).

To find n_j^* , we use the equilibrium condition: arrivals must equal completions. Each iteration processes one stage for all n_j^* prompts, taking time $(d_0 + d_1 M_j^*)$. During this time, $\lambda_j (d_0 + d_1 M_j^*)$ new prompts arrive. For equilibrium, this must equal the $n_j^*/(l'_j + 1)$ prompts that complete:

$$\underbrace{(d_0 + d_1 M_j^*)}_{\text{Time per iteration}} \lambda_j = \left(d_0 + d_1 n_j^* \left(l_j + \frac{l'_j}{2} \right) \right) \lambda_j = \frac{n_j^*}{l'_j + 1}.$$

Solving for n_j^* yields:

$$n_j^* = \frac{d_0 \lambda_j (l'_j + 1)}{1 - d_1 \lambda_j (l'_j + 1) (l_j + \frac{l'_j}{2})}.$$

Substituting this into the memory equation, the equilibrium memory equals:

$$M_j^* = n_j^* \left(l_j + \frac{l'_j}{2} \right) = \frac{d_0 \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}{1 - d_1 \lambda_j (l'_j + 1) (l_j + \frac{l'_j}{2})}. \quad (5)$$

This equilibrium batch size balances arrivals and completions. The denominator $(1 - d_1 \lambda_j (l'_j + 1) (l_j + l'_j/2))$ represents the fraction of iteration time available for processing—as this approaches zero, batch size grows unbounded (approaching instability). The average throughput at equilibrium is the number of decode tokens generated per iteration divided by the time consumption for one iteration. In each iteration, $n_j^*/(l'_j + 1)$ prompts complete (those at stage l'_j), each generating l'_j decode tokens:

ATTENTION: The following displayed equation, in its current form, exceeds the column width that will be used in the published edition of your article. Please break or rewrite this equation to fit, including the equation number, within a column width of 240 pt / 84.67 mm / 3.33 in (the width of this red box).

$$\text{Throughput}_j^* = \frac{n_j^* \cdot l'_j}{(l'_j + 1) (d_0 + d_1 n_j^* (l_j + \frac{l'_j}{2}))} = \lambda_j l'_j, \quad (6)$$

where the equality follows from the equilibrium condition in line 46. This shows how arrival rate, memory, and throughput relate in equilibrium, where arrivals equal completions.

3.2. Multiple-Type Fluid Model

The single-type analysis provides intuition, but real systems handle multiple prompt types simultaneously, each with different input lengths l_j and output lengths l'_j . This heterogeneity introduces a key challenge: types compete for the same memory capacity C . Admitting a long prompt (large l'_j) may force evicting multiple short prompts, wasting their partial computation—a cascading failure we analyze in Section 4. We now extend the equilibrium model to capture this multi-type interaction.

We assume the system maintains n_j^* active prompts of each type $j \in \{1, \dots, m\}$ in equilibrium. Following the derivation of (5), the total memory occupancy equals:

$$M^* = \sum_{j=1}^m n_j^* \left(l_j + \frac{l'_j}{2} \right) \quad (7)$$

with:

$$\underbrace{(d_0 + d_1 M^*)}_{\text{Time per iteration}} \lambda_j (l'_j + 1) = n_j^*, \forall j \in \{1, 2, \dots, m\}$$

since the arrivals during each iteration should equal the number of completions in equilibrium, and there are $(l'_j + 1)$ stages total. Substituting the above equations into (7) yields

$$M^* = \frac{d_0 \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}{1 - d_1 \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)} \quad (8)$$

Aggregating the results above, we get the processing time per iteration in equilibrium:

$$\begin{aligned} & \text{Processing Time}^* \\ &= d_0 + d_1 M^* \\ &= \frac{d_0}{1 - d_1 \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}, \end{aligned} \quad (9)$$

as well as the average throughput:

ATTENTION: The following displayed equation, in its current form, exceeds the column width that will be used in the published edition of your article. Please break or rewrite this equation to fit, including the equation number, within a column width of 240 pt / 84.67 mm / 3.33 in (the width of this red box).

$$\mathbf{Throughput}^* = \sum_{j=1}^m \frac{n_j^* \cdot l'_j}{(l'_j + 1) \cdot (d_0 + d_1 M^*)} = \sum_{j=1}^m \lambda_j l'_j \quad (10)$$

The equilibrium throughput benchmarks all non-predictive policies within the policy class Π . The following proposition demonstrates that the expected throughput of any policy in Π cannot exceed the equilibrium throughput $\mathbf{Throughput}^*$, as specified in Equation (10).

PROPOSITION 2. *Suppose that the memory capacity satisfies $C \geq M^*$, where M^* denotes the equilibrium memory given in (8). Then, for any online policy $\pi \in \Pi$, the expected throughput satisfies*

$$\mathbb{E}[\mathbf{Throughput}^{(T,\pi)}] \leq \mathbf{Throughput}^*,$$

where $\mathbf{Throughput}^*$ is given in (10).

This result has two implications for algorithm design. First, it quantifies the theoretical limit: no non-predictive policy can exceed $\mathbf{Throughput}^* = \sum_{j=1}^m \lambda_j l'_j$ when $C \geq M^*$. This establishes a performance benchmark for any scheduling policy. Second, it suggests a design principle: any policy that prevents evictions and maintains balanced stages can approach this throughput. This motivates our WAIT algorithm (Section 4), which uses thresholds to keep the system near equilibrium, preventing the cascading failures that cause FCFS to fall short (see Example 2).

The equilibrium analysis reveals a critical design principle: maintain balanced prompt distribution across stages. Having exactly $n_j^*/(l'_j + 1)$ prompts at each stage prevents two failure modes. First, if prompts accumulate at later decode stages—where each consumes more memory—total usage may exceed capacity C , forcing evictions that waste partial computation. Second, if prompts concentrate at early stages while later stages starve, the system leaves processing capacity unused, reducing throughput.

This load-balancing principle directly motivates our WAIT algorithm (Section 4). By controlling when prompts advance to the next stage via thresholds, WAIT keeps the system close to the balanced equilibrium, preventing the cascading evictions that plague FCFS (see Example 2 in Section 4). The fluid model thus provides both a performance benchmark and a design guide.

In classical queueing theory, fluid models typically lead to ordinary differential equations (ODEs) describing the continuous transition of queue lengths over time. Constructing such ODEs for LLM inference, however, presents technical challenges. Eviction events—triggered when memory usage exceeds capacity—cause discontinuous jumps in the system state, as KV caches of evicted prompts are cleared entirely rather than decreasing smoothly. At boundaries where certain stage populations reach zero or where drift terms vanish, the direction of state transition becomes ambiguous under standard ODE analysis. We therefore adopt a discrete, iteration-based framework that directly characterizes equilibrium and uses it as a performance benchmark pending a complete characterization of such an asymptotic regime.

4. Scheduling with Known Arrival Types: The WAIT Algorithm

This section introduces the *Waiting for Accumulated Inference Threshold* (WAIT) algorithm, a scheduling policy that maximizes throughput under known prompt arrival types. Using the fluid dynamics established in Section 3, WAIT optimizes batch formation and scheduling by accumulating prompts until specific thresholds are met. This approach uses resources efficiently in stochastic environments by maintaining the system near equilibrium while respecting memory constraints and optimizing latency and time-to-first-token (TTFT). We assume that the memory capacity $C \geq M^*$, the equilibrium memory capacity given in (8) (otherwise the throughput can never approach the optimal fluid throughput (10) by Proposition 2).

REMARK 1 (ON THE ASSUMPTION $C \geq M^*$). Our theoretical analysis assumes $C \geq M^*$, as this is the necessary condition under which the system can achieve stable operation without accumulating backlog. When $C < M^*$ (the overloaded regime), our algorithms remain effective: the eviction prevention mechanism still outperforms baselines, as demonstrated in Section 6. Avoiding eviction cascades provides value across all operating regimes.

4.1. Motivation: Why Naive Scheduling Fails

We first demonstrate why naive scheduling policies fail even when memory capacity is sufficient, motivating the design of WAIT.

Consider what happens under a First-Come-First-Serve (FCFS) policy that processes prompts immediately upon arrival. Even when the memory capacity C equals the equilibrium memory M^* —seemingly sufficient to handle the workload—FCFS can trigger catastrophic failure. The following proposition formalizes this observation.

PROPOSITION 3. *When the memory capacity C equals the equilibrium memory M^* , there exists an instance where the First-Come-First-Serve (FCFS) policy incurs a throughput gap of $\text{Throughput}^* - \mathbb{E}[\text{Throughput}^T] = \Omega(1)$ as $T \rightarrow \infty$.*

The proof is provided in Appendix. To illustrate the mechanism behind this failure concretely, consider the following example.

EXAMPLE 2 (EVICTION CASCADE UNDER FCFS). Consider a single prompt type where each prompt is a simple query such as “Hello” with a 1-token response “Hi”—prefill length $l = 1$ and decode length $l' = 1$. Each prompt occupies 1 unit of memory after prefill and 2 units after decode. Let the memory capacity be $C = 12$ and arrival rate $\lambda = 4$. The fluid equilibrium has 4 prompts at each stage (prefill and decode), using exactly $4 \times 1 + 4 \times 2 = 12$ units of memory with 4 completions per iteration.

Now suppose the system starts slightly imbalanced with 6 prompts at stage 0 (awaiting prefill) and 3 prompts at stage 1 (in decode). Under FCFS:

1. **Iteration 1:** Process all 9 prompts. The 3 decode prompts complete, but the 6 prefill prompts advance to decode, requiring $6 \times 2 = 12$ units. Memory is exactly full with only 3 completions.
2. **Iteration 2:** During iteration 1, approximately 4 new prompts arrive. To admit them for prefill ($4 \times 1 = 4$ units needed), we must evict 2 decode prompts (freeing $2 \times 2 = 4$ units). Now we have 4 prefill + 4 decode = $4 + 8 = 12$ units.
3. **Iteration 3:** The 4 decode prompts complete (4 completions). The 2 previously evicted prompts, having lost their KV caches, must now restart from the prefill phase (stage $s = 0$)—they re-enter the queue as new arrivals, competing for resources alongside genuinely new prompts. This restart mechanism perpetuates the imbalance.

This eviction-restart cycle persists, yielding approximately 3-3.5 completions per iteration instead of the optimal 4. **Even though $C = M^* = 12$ (theoretically sufficient capacity), FCFS causes cascading failures that reduce throughput by 12-25%.**

This example demonstrates a fundamental insight: *memory sufficiency alone does not guarantee stability*—a system that should be stable can become unstable under poor scheduling. The root cause is that FCFS does not maintain *load balance*: it admits prompts greedily, causing the system to deviate from the fluid equilibrium and triggering the eviction-restart cycle.

4.2. The WAIT Algorithm

The WAIT algorithm prevents eviction cascades by maintaining load balance through a threshold mechanism. The core idea is to control admission at each decode stage, ensuring that the rate of new arrivals matches the rate of completions. This keeps the system close to the fluid equilibrium derived in Section 3, *approaching load balance* where arrival and completion rates match. This principle—maintaining the system near equilibrium through controlled admission—is central to preventing the cascading failures illustrated in Example 2.

The threshold-and-wait mechanism achieves three goals. First, it prevents *eviction cascades* by controlling admission. Second, it *decouples the complex multi-stage, multi-type dynamics*, transforming the system into m independent single-type analyses. Third, by waiting until thresholds are met, the batch approaches the fluid equilibrium size, *using memory capacity fully to maximize throughput*. The decoupling is crucial: without thresholds, delays in one type would affect memory availability for others through shared capacity constraints, creating feedback loops that resist standard queueing analysis. The threshold mechanism breaks these inter-type dependencies through algorithm design, not inherent problem simplicity. The third goal reflects a key insight: memory is not just a constraint to satisfy, but a resource to exploit—larger batches process more prompts per iteration, and waiting for the threshold ensures we form batches large enough to use memory efficiently while staying within safe limits.

Algorithm Description. For each prompt type $j \in [m]$, WAIT sets a threshold n_j derived from the fluid dynamics. At time t , the algorithm monitors the inventory n_{js}^t —the number of waiting prompts of type j at stage s . Type j prompts are included in a batch only when:

$$n_{j0}^t \geq n_j.$$

When this condition is met, WAIT constructs a batch by selecting $\min\{n_j, n_{js}^t\}$ prompts of type j at each stage s for all types meeting the threshold. If no type satisfies the condition, the algorithm waits for more arrivals. KV caches for waiting prompts are preserved, avoiding redundant computation. The complete specification is in Algorithm 1 and Figure 4.

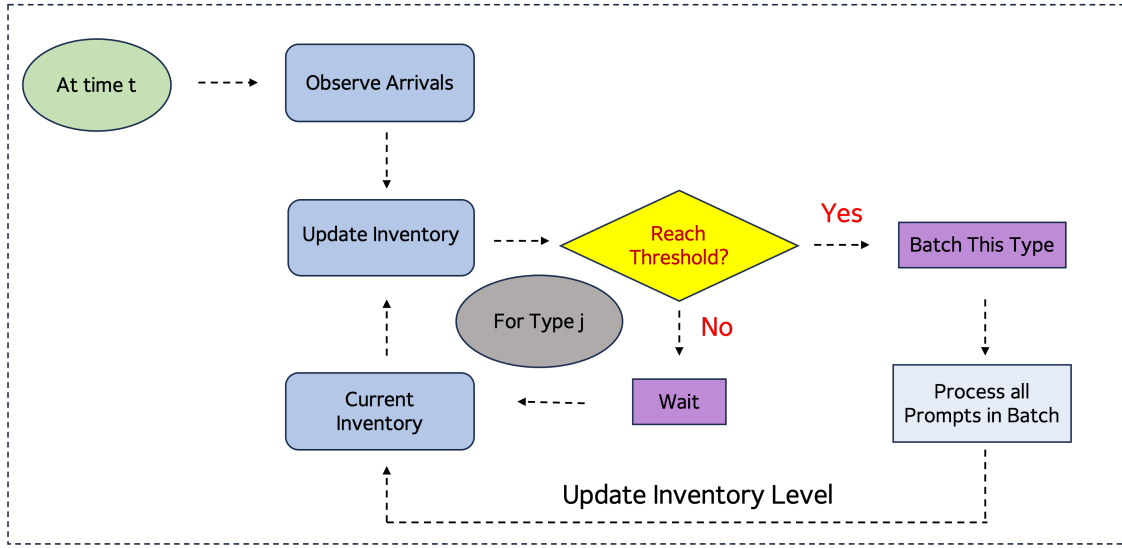


Figure 4 Pipeline of Algorithm 1

Algorithm 1 WAIT: Waiting for Accumulated Inference Threshold

Input: Memory C , arrival rates λ_j , thresholds n_j satisfying (11), $\forall j \in [m]$

- 1: Initialize prompt inventory $n_{js} \leftarrow 0$ for all $j \in [m], s \in \{0, 1, \dots, l'_j\}$
 - 2: Initialize event queue with arrival events for each type j
 - 3: Set current time $t \leftarrow 0$
 - 4: **while** True **do**
 - 5: Wait for the next event (arrival or batch completion)
 - 6: **if** event is an arrival of type j **then**
 - 7: Update inventory: $n_{j0} \leftarrow n_{j0} + 1$ ▷ Add new prompt to waiting queue of prefill phase
 - 8: **else if** event is a batch completion **then**
 - 9: Update inventory: For each j in batch B , $n_{js} \leftarrow n_{js} - \min\{n_j, n_{js}\}$ for $s = 0, \dots, l'_j$
 - 10: Advance prompts: For each j , move $\min\{n_j, n_{js}\}$ prompts from stage s to $s + 1$
 - 11: Clear KV caches for prompts reaching stage $l'_j + 1$ ▷ Free memory for completed prompts
 - 12: **end if**
 - 13: Check if $n_{j0} \geq n_j$ for any $j \in [m]$ ▷ Check threshold for batching
 - 14: **if** condition $n_{j0} \geq n_j$ is met for some j **then**
 - 15: Form batch B by selecting $\min\{n_j, n_{js}\}$ prompts of type j at each stage s for all j meeting the condition
 - 16: Process batch B , keep prompts outside the batch waiting and do not delete their KV caches
 - 17: **end if**
 - 18: **end while**
-

4.3. Asymptotic Analysis

We evaluate the performance of WAIT in an asymptotic regime that captures long-run system behavior. Specifically, we scale the arrival rates to $\lambda_j^{(\zeta)} = \zeta \lambda_j$ and processing speeds accordingly $((d_0^{(\zeta)}, d_1^{(\zeta)}) = (\zeta^{-1} d_0, \zeta^{-1} d_1))$ while keeping memory capacity C fixed. This can equivalently be viewed as scaling the time horizon from T to ζT while keeping arrival rates and processing speeds unchanged. As $\zeta \rightarrow \infty$, we obtain infinite-horizon limits that characterize the system's long-run average performance, allowing us to study throughput stability under sustained load.

Performance Metric: We focus on the average throughput, denoted $\mathbf{Throughput}^{(\zeta, \pi)}$, where π represents the WAIT policy, defined as:

$$\mathbf{Throughput}^{(\zeta, \pi)} = \frac{1}{\zeta T} \sum_{j=1}^m N_j^{(\zeta, \pi)},$$

with $N_j^{(\zeta, \pi)}$ being the total number of type- j tokens processed under scaling ζ . The average latency $\mathbf{Latency}^{(\zeta, \pi)}$, $\mathbf{TTFT}^{(\zeta, \pi)}$ and TTFT are defined similarly by dividing the quantity by ζ . Our objective is to demonstrate that $\mathbf{Throughput}^{(\zeta, \pi)}$ approaches the fluid equilibrium benchmark $\mathbf{Throughput}^* = \sum_{j=1}^m \lambda_j l'_j$ (10) as $\zeta \rightarrow \infty$ while giving bound to the latency and TTFT.

4.4. Theoretical Analysis

When memory capacity satisfies $C \geq M^*$ (where M^* is the equilibrium memory from Equation (8)), consider the WAIT policy π parameterized by thresholds $[n_1, \dots, n_m]$, where n_j is the threshold for type- j prompts at each stage. WAIT is asymptotically optimal when the thresholds satisfy:

$$\begin{aligned} \Delta T(n_{1:m}) &:= d_0 + d_1 M^\pi \leq \frac{n_j}{\lambda_j}, \forall j \in [m], \\ M^\pi &= \sum_{j=1}^m n_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right) \end{aligned} \quad (11)$$

The intuition follows from Equation (9): when type j is included in a batch, the iteration time ΔT ensures that expected arrivals of type j do not exceed n_j . This keeps the system close to the fluid equilibrium, using memory capacity fully for throughput while avoiding overflow.

THEOREM 1. *Assume the thresholds $\pi = [n_1, \dots, n_m]$ satisfy (11), then we have*

$$\begin{aligned} \mathbf{Throughput}^* - \mathbb{E} [\mathbf{Throughput}^{(\zeta, \pi)}] &= O((\zeta T)^{-\frac{1}{2}}), \\ \mathbb{E} [\mathbf{Latency}^{(\zeta, \pi)}], \mathbb{E} [\mathbf{TTFT}^{(\zeta, \pi)}] &= O((\zeta T)^{\frac{1}{2}}). \end{aligned}$$

Moreover, when we have $\Delta T_{[1, \dots, m]} < n_j / \lambda_j, \forall j \in [m]$ where $\Delta T_{[1, \dots, m]}$ is defined in (11), it holds that

$$\begin{aligned} \mathbf{Throughput}^* - \mathbb{E} [\mathbf{Throughput}^{(\zeta, \pi)}] &= O(1/(\zeta T)), \\ \mathbb{E} [\mathbf{Latency}^{(\zeta, \pi)}], \mathbb{E} [\mathbf{TTFT}^{(\zeta, \pi)}] &= O(1). \end{aligned}$$

The WAIT algorithm prevents eviction cascades through a simple but powerful principle: controlled admission via thresholds. By requiring $n_{j0}^t \geq n_j$ before admitting type- j prompts, the algorithm ensures that expected arrivals during each iteration do not exceed completions. This

keeps each prompt type close to its equilibrium inventory level, preventing the overflow that triggers eviction cascades in FCFS (Example 2).

The threshold mechanism simultaneously achieves three design goals: (1) *eviction prevention* by controlling when prompts enter batches; (2) *approaching load balance* by maintaining the system near the fluid equilibrium where arrivals match completions; and (3) *memory exploitation* by waiting until batches can be formed close to the equilibrium size M^* , maximizing throughput.

From an analytical perspective, the threshold decouples the m prompt types: we can analyze each type independently, as if it were the only type in the system. Without this decoupling, delays in one type would affect memory availability for others through the shared capacity constraint, creating feedback loops that resist standard queueing analysis. With types decoupled, we apply standard techniques—constructing dominating processes and using drift analysis—to establish the asymptotic optimality bounds. The key insight: it is the algorithm design, not inherent problem simplicity, that makes this tractable. The complete proof is in Appendix B.

Theorem 1 provides performance bounds that match the worst-case lower bounds for latency and Time to First Token (TTFT). The following proposition establishes these lower bounds.

PROPOSITION 4. *There exists an instance where $C = M^*$, and for any non-predictive online policy π , the expected average latency and TTFT satisfy*

$$\mathbb{E}[\mathbf{Latency}^{(\zeta, \pi)}], \quad \mathbb{E}[\mathbf{TTFT}^{(\zeta, \pi)}] = \Omega((\zeta T)^{1/2}).$$

Moreover, we demonstrate that the latency for type j prompts depends exclusively on the relationship between $\Delta T_{[1, \dots, m]}$ and n_j/λ_j . Specifically, when $\Delta T_{[1, \dots, m]} < n_j/\lambda_j$, the average latency and TTFT for type j prompts are $O(1)$, offering a more precise performance estimate than that provided in Theorem 1.

A limitation of WAIT is its reliance on known arrival types and rates, which may not hold in realistic environments (though we can approximate it by using predictors, see e.g. (Fu et al. 2025)). In the next section, we address this issue by extending WAIT to handle unknown arrival types, introducing a nested variant that adapts to stochastic settings with unknown prompt types upon arrival.

5. Scheduling with Unknown Arrival Types: The Nested WAIT Algorithm

The WAIT algorithm achieves near-optimal throughput when we know prompt types at arrival. In practice, however, we cannot predict how many tokens a prompt will generate—output lengths reveal themselves only as decode progresses. The *Nested Waiting for Accumulated Inference Threshold* (Nested WAIT) algorithm exploits this gradual revelation: **prompts classify themselves on-the-fly** as they decode. Short prompts complete early and exit; longer prompts naturally advance to later segments. **No prediction is needed**—the algorithm simply lets each prompt reveal its type through its decode trajectory.

EXAMPLE 3 (UNKNOWN OUTPUT LENGTHS). Continuing Example 2, suppose output lengths are now *unknown* at arrival: each “Hello” prompt receives either “Hi” (1 token) or “Hi there!” (2 tokens). The scheduler faces a dilemma. If it optimistically assumes all outputs are short (1 token each), it admits 6 prompts with capacity $C = 12$, requiring $6 \times 2 = 12$ units after decode. But if some outputs turn out to be 2 tokens (requiring 3 units each), memory overflows and eviction cascades begin. Conversely, if the scheduler conservatively assumes all outputs are long, it admits only 4 prompts to stay safe, wasting 25% of capacity when most outputs are actually short.

This dilemma—optimism leads to eviction, pessimism wastes capacity—leads to our design principles: (i) *avoid eviction* by not over-committing memory; (ii) *classify on-the-fly* by letting prompts reveal their type as they decode; (iii) *don't waste throughput* by exploiting available capacity; (iv) *let shorter prompts finish first* so they free memory without being blocked by longer ones.

5.1. Algorithm Overview

Example 3 reveals a fundamental dilemma: optimism risks eviction, pessimism wastes capacity. We resolve this through on-the-fly classification. Rather than predicting output lengths at arrival, Nested WAIT lets prompts reveal their output as they decode. Short prompts complete early and exit; longer prompts naturally advance to later segments.

The core idea is **nested thresholds**: segment 1 handles all prompts from decode stage 0 to l'_1 ; segment 2 handles prompts that survive past l'_1 (i.e., types 2 through m); segment 3 handles prompts past l'_2 ; and so on. Each segment k has its own threshold n_k controlling when it processes prompts. This nesting achieves two goals: (i) short prompts complete in early segments without being blocked by long prompts; (ii) each segment operates at high memory utilization for its remaining types.

Formally, let output lengths satisfy $l'_1 < l'_2 < \dots < l'_m$, where type j has decode length l'_j . We organize inference into m nested segments, where segment k spans decode stages from $l'_{k-1} + 1$ to l'_k (with $l'_0 = 0$). The algorithm uses thresholds $\pi = [n_1, \dots, n_m]$ where n_k controls when segment k begins processing and limits batch size per stage.

Segment 1 processes all newly arrived prompts (at combined rate $\lambda_1 + \dots + \lambda_m$) through decode stages $0, 1, \dots, l'_1$. After $l'_1 + 1$ iterations, two outcomes emerge: type-1 prompts have completed their output and **exit the system**, freeing their memory; all other prompts (types 2 through m) **advance to segment 2**. Segment 2 now processes only these longer prompts (at reduced rate $\lambda_2 + \dots + \lambda_m$) through stages $l'_1 + 1, \dots, l'_2$. This pattern repeats: after $l'_2 - l'_1$ more iterations, type-2 prompts exit, and types 3 through m advance to segment 3. **Prompts classify themselves** by completing (short) or continuing (long) at each segment boundary.

If segment k^* has fewer than n_{k^*} prompts, we pause processing for that segment and all later ones, retaining their KV caches on GPU. We continue batching exactly n_k prompts per stage in segments $k < k^*$ and perform iterations. Since new long prompts arrive continuously, segment k^* resumes once its threshold is met. Algorithm 2 formalizes this procedure, and Figure 5 illustrates the pipeline.

We assume identical prefill lengths $l_1 = l_2 = \dots = l_m$ for clarity, though the general case extends similarly to Algorithm 1 by forming segment groups for each prefill length. The number of output lengths m minimally impacts performance, which depends primarily on total arrival rates. The number of segments can also differ from m while retaining near-optimal guarantees (Section 7).

Consider Example 3 with two prompt types: “Hello” $\rightarrow$ “Hi” (1 token, type 1) and “Hello” $\rightarrow$ “Hi there!” (2 tokens, type 2). At arrival, we cannot distinguish them. After one decode iteration, however, **the prompts reveal themselves**: type-1 prompts have completed their output (“Hi”) and exit; type-2 prompts have generated only their first token (“Hi”) and continue. Nested WAIT exploits this revelation by partitioning processing into two segments. **Segment 1** processes all newly arrived prompts (both types) through stages $0, 1$ (up to $l'_1 = 1$) at combined rate $\lambda_1 + \lambda_2$. **Segment 2** processes only type-2 prompts (those that survived segment 1) through stage 2 (up to $l'_2 = 2$) at reduced rate λ_2 . **No prediction is needed**—prompts simply classify themselves by completing or continuing at the segment 1 boundary.

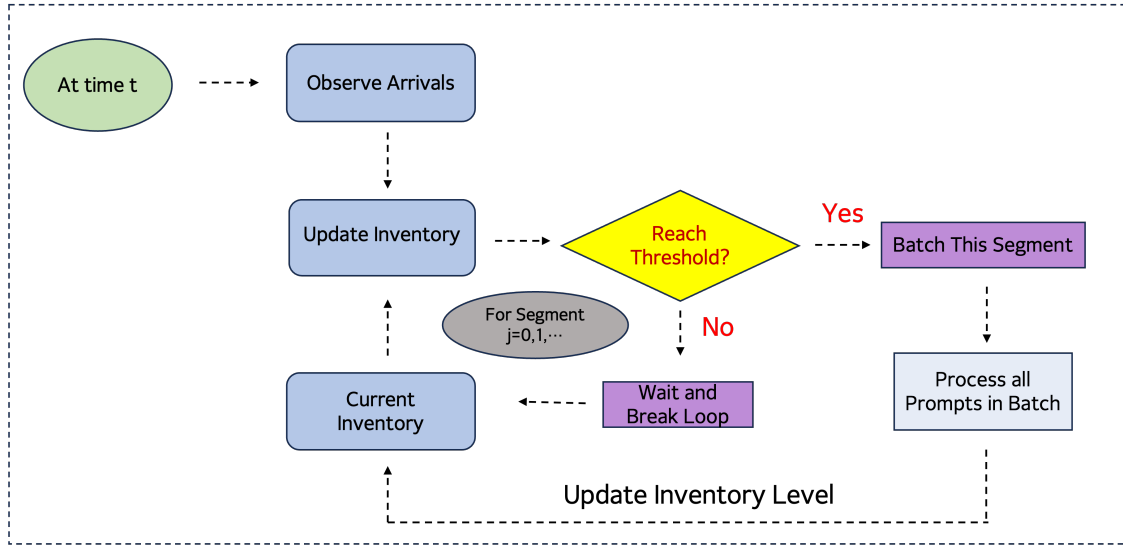


Figure 5 Pipeline of the Nested WAIT Algorithm

Figure 6 visualizes the algorithm in action. At time $t = 0$, prompts $P1$ and $P2$ arrive but are insufficient to trigger processing, as Segment 1 has fewer than 3 prompts. At $t = 1$, $P3$ arrives, satisfying the Segment 1 threshold, and the batch is processed. Since these prompts have just begun decoding, none qualify for Segment 2. At $t = 5$, a new wave of prompts ($P4$, $P5$) arrives, again below the threshold. By $t = 7$, the threshold is met with $P6$, $P7$, and processing proceeds. Now, $P3$, having reached a deeper decode stage, transitions to Segment 2. However, since the Segment 2 threshold is still unmet, only Segment 1 is processed. Finally, at $t = 14$, enough prompts have advanced into Segment 2 ($P3$, $P5$, $P6$, etc.) to satisfy both segment thresholds. At this point, the Nested WAIT algorithm jointly schedules both segments, batching according to type-specific readiness. This example shows how the algorithm defers processing to ensure type separation and stage-aligned batching.

This nested structure directly implements the four design principles from Example 3: (i) we **avoid eviction** by never exceeding thresholds; (ii) prompts **classify on-the-fly** at segment boundaries; (iii) we **don't waste throughput** because each segment operates at its own optimal threshold; (iv) **short prompts finish first** within segment 1, freeing memory without waiting for long prompts. Crucially, **no prediction is needed**—prompts simply reveal their output by completing or continuing. This achieves high memory utilization **for all types simultaneously**, not just the shortest or longest. We now formalize this intuition.

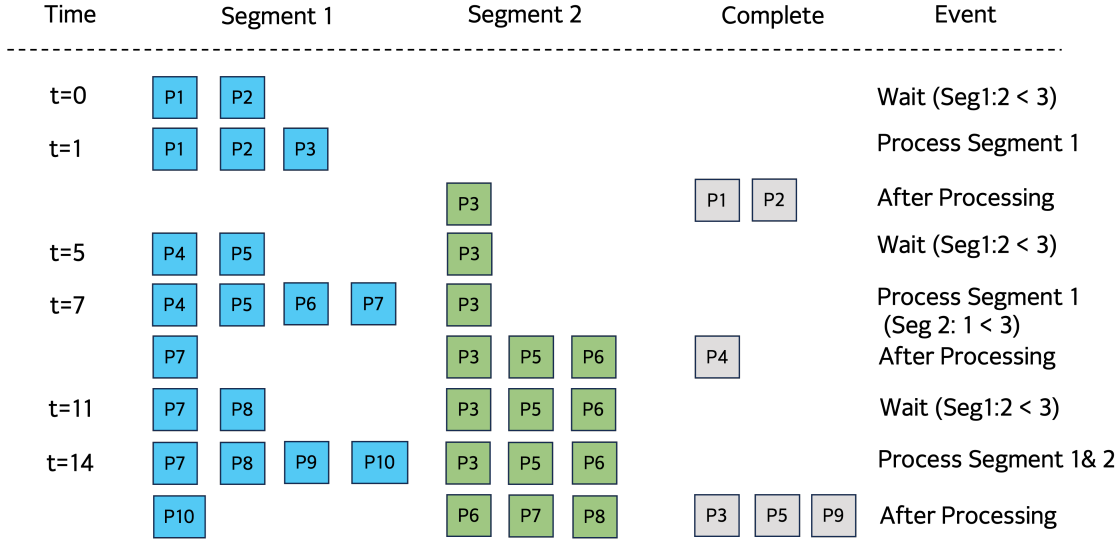


Figure 6 Example of the Nested WAIT Algorithm

Algorithm 2 Nested WAIT: Nested Waiting for Accumulated Inference Threshold

Input: Memory capacity C , arrival rates λ_j for $j \in [m]$, thresholds n_k for segments $k \in [m]$

Input: Output lengths $l'_1 < l'_2 < \dots < l'_m$ for prompt types $j \in [m]$

$\triangleright Q_{k,s}$: number of prompts in segment k at decode stage s ; entry stage of segment k is l'_{k-1} (with $l'_0 = 0$)

- 1: Initialize $Q_{k,s} \leftarrow 0$ for all segments $k \in [m]$ and stages $s \in [l'_{k-1}, l'_k - 1]$
- 2: Initialize event queue; set current time $t \leftarrow 0$
- 3: **while** True **do**
- 4: Wait for the next event (arrival or batch completion)
- 5: **if** event is an arrival **then**
- 6: $Q_{1,0} \leftarrow Q_{1,0} + 1$ $\triangleright$ New prompts enter segment 1 at stage 0
- 7: **else if** event is a batch completion for segment k **then**
- 8: **for** each stage s in segment k **do**
- 9: Advance $\min\{n_k, Q_{k,s}\}$ prompts from stage s to $s + 1$
- 10: **end for**
- 11: **if** prompts reach stage l'_k (end of segment k) **then**
- 12: **if** $k < m$ **then**
- 13: Move to segment $k + 1$ at stage l'_k $\triangleright$ These are type- $(k + 1)$ or longer
- 14: **else**
- 15: Complete and clear KV caches
- 16: **end if**
- 17: **end if**
- 18: **end if**
- 19: Find largest k such that $Q_{k',l'_{k'-1}} \geq n_{k'}$ for all $k' \leq k$ $\triangleright$ All segments up to k meet threshold
- 20: **if** such k exists **then**
- 21: Form batch from segments $1, \dots, k$: select $\min\{n_{k'}, Q_{k',s}\}$ prompts per stage
- 22: Process batch; prompts outside batch wait with KV caches retained
- 23: **end if**
- 24: **end while**

5.2. Main Theorem

We now analyze Nested WAIT in the asymptotic regime, following the same framework as Section 4. The unknown-output-length setting introduces **additional stochastic variability**: since we cannot predict which prompts will be long, some may queue at segment boundaries waiting for thresholds. This affects both memory requirements and throughput guarantees. We characterize average throughput $\text{Throughput}^{(\zeta, \pi)}$, latency $\text{Latency}^{(\zeta, \pi)}$, and time-to-first-token $\text{TTFT}^{(\zeta, \pi)}$ under threshold policy $\pi = [n_1, \dots, n_m]$.

Before showing Nested WAIT achieves near-optimal throughput, we establish a lower bound showing that **some** extra memory beyond M^* is necessary when output lengths are unknown.

Intuition: When memory capacity equals exactly $C = M^*$ (the fluid equilibrium), any algorithm that cannot predict output lengths faces a dilemma at every iteration. If we admit too many prompts optimistically, some will reveal longer outputs than expected, causing overflow and eviction. If we admit too few conservatively, we waste capacity. Either scenario creates a throughput gap of $\Omega(1)$ over time.

PROPOSITION 5. *When memory capacity C equals the equilibrium memory M^* , there exists an instance where any online non-predictive policy π , unaware of prompt types upon arrival, incurs a throughput gap $\text{Throughput}^* - \mathbb{E}[\text{Throughput}^{(\zeta, \pi)}] = \Omega(1)$.*

This lower bound shows that uncertainty has a cost: we need more memory than the fluid equilibrium to hedge against unknown output lengths. Theorem 2 shows that only a **logarithmic** amount of extra memory suffices under Nested WAIT.

As in Section 4, thresholds must balance two goals: (i) **throughput efficiency** (don't wait too long to form batches); (ii) **segment stability** (ensure later segments receive enough prompts). This leads to two constraints, analogous to Equation (11).

Similar to Equation (11), we define $\Delta T_{[1, \dots, m]}(n_1, \dots, n_m)$ as the time cost per iteration with exactly n_k prompts per stage in segment k , for all $k \in [1, \dots, m]$. Thresholds must satisfy:

$$\begin{aligned} \Delta T_{[1, \dots, m]}(n_1, \dots, n_m) &< \frac{n_1}{\sum_{j=1}^m \lambda_j}, \\ \frac{n_{k+1}}{n_k} &> p_k, \forall k \in \{1, \dots, m-1\}, \end{aligned} \tag{12}$$

where $p_k = (\sum_{j=k+1}^m \lambda_j) / (\sum_{j=k}^m \lambda_j) < 1$. To state the memory requirement, we define the memory usage when running iteration with exactly n_k prompts per stage in segment k , for all $k \in [1, \dots, m]$ as:

$$M^\pi = \sum_{k=1}^m n_k \left(l + \frac{1}{2} L'_k \right) \Delta l'_k, \quad (13)$$

$$\text{where } L'_k = \sum_{r=1}^k l'_r, \Delta l'_k = l'_k - l'_{k-1}, l'_0 = 0.$$

Here, the formula captures the peak batch memory when exactly n_k prompts occupy each segment k : the term n_k is the threshold (maximum prompts concurrently decoding in segment k), $\Delta l'_k = l'_k - l'_{k-1}$ counts the decode stages spanned by segment k , and $(l + \frac{1}{2} L'_k)$ represents the average memory per prompt within segment k , where $L'_k = l'_1 + \dots + l'_k$ is the cumulative decode length. We define the total number of iterations over time horizon $[0, T]$ as B . Therefore, we have $T \geq \sum_{b=1}^B \Delta T_{b\text{-th Batch}} \geq B \cdot \min \Delta T_{\text{Batch}}$, where $\Delta T_{b\text{-th Batch}}$ denotes the time cost of the b -th iteration and $\Delta T_{\min} = \min_b \{\Delta T_{\text{Batch}}\}$. Theorem 2 establishes asymptotic optimality for Nested WAIT in the asymptotic regime with unknown prompt types, given thresholds $\pi = [n_1, \dots, n_m]$ satisfying Equation (12).

THEOREM 2. Assume thresholds $\pi = [n_1, \dots, n_m]$ satisfy Equation (12), and memory M satisfies:

$$\begin{aligned} M^{(\zeta, \pi)} &\geq M^\pi + \sum_{k=2}^m (l + l'_{k-1}) \left(n_k + \theta_k^{-1} \ln \left(\frac{m\zeta B}{\delta} \right) \right) \\ &= O \left(2M^\pi + \sum_{k=1}^m (l + l'_{k-1}) \theta_k^{-1} \ln \left(\frac{m\zeta B}{\delta} \right) \right), \end{aligned} \quad (14)$$

where $\theta_k \geq 8(n_k - n_{k-1}p_k)/n_k$. Then:

$$\begin{aligned} \text{Throughput}^* - \mathbb{E}[\text{Throughput}^{(\zeta, \pi)}] &= O((\zeta T)^{-1}), \\ \mathbb{E}[\text{Latency}^{(\zeta, \pi)}], \mathbb{E}[\text{TFT}^{(\zeta, \pi)}] &= O(1). \end{aligned}$$

Moreover, memory is not exceeded with probability at least $1 - \delta$ during time horizon $[0, T]$.

Nested WAIT extends the dimensionality reduction approach from WAIT to handle unknown output lengths. The core insight is that thresholds n_k in each segment k bound the state space, just as in WAIT—but now prompts classify themselves on-the-fly by completing or continuing at segment boundaries, rather than revealing their type at arrival. There is even no need to know

the actual output—we only observe whether each prompt completes or continues. This on-the-fly classification creates a key technical challenge: prompts entering segment 2 at time t are precisely those that survived segment 1, creating a delayed arrival process with rate $\lambda_2 + \dots + \lambda_m$ that depends on the history of earlier segments. We address this through a coupling argument: the number of prompts transitioning from segment k to $k + 1$ couples to a Poisson process with rate $\sum_{j=k+1}^m \lambda_j$, allowing us to analyze each segment's queue independently despite the temporal dependency.

This coupling reveals why the safety buffer is necessary. When segment k has fewer than n_k prompts, processing pauses and prompts queue at the boundary. Because we cannot predict which prompts will advance to later segments, this queueing introduces stochastic fluctuations beyond the fluid limit. Doob's maximal inequality bounds the maximum queue length over time horizon $[0, T]$ with high probability, showing the bound grows only as $O(\ln(T/\delta))$ —this logarithmic overhead is the safety buffer in Equation (14). The key is that despite not knowing output lengths, the threshold mechanism keeps each segment close to its equilibrium batch size, maintaining the system near the load-balanced state where arrivals match completions. This prevents the cascading evictions that occur in FCFS (Example 2) while achieving the same design goals as WAIT: eviction prevention, approaching load balance, and memory exploitation. The full proof in Appendix C formalizes the coupling construction and derives explicit constants.

The memory requirement in Equation (14) consists of two components with clear operational interpretations:

1. **Base memory** M^π : This is the equilibrium memory from the fluid model—the memory needed when exactly n_k prompts occupy each stage in segment k . This represents the “steady-state” memory consumption.

2. **Safety buffer** $\sum_{k=2}^m (l + l'_{k-1})(n_k + \theta_k^{-1} \ln(m\zeta B/\delta))$: This additional memory hedges against stochastic fluctuations when output lengths are unknown. It has three components:

- n_k : Accounts for prompts queueing at segment k boundaries, waiting for thresholds.
- $\theta_k^{-1} \ln(m\zeta B/\delta)$: High-probability protection against queue buildup (overflow occurs with probability $\leq \delta$).
- $(l + l'_{k-1})$: Memory footprint of prompts at the boundary between segments $k - 1$ and k .

This safety buffer is *logarithmic* in the time horizon T (via B) and confidence level $1/\delta$, making it a modest overhead compared to the base memory M^π . This is why Proposition 5 shows that *some*

extra memory beyond M^* is necessary for unknown output lengths, but our Theorem 2 demonstrates that only a logarithmic amount suffices.

Number of types: The algorithm's performance depends primarily on arrival rates and output lengths, not the number of types m . Even for large m (e.g., output lengths $\{1, 2, \dots, m\}$), descending thresholds $n_1 > n_2 > \dots > n_m$ remain effective when arrival rates satisfy condition (12).

Safety buffer overhead: The memory requirement in (14) is typically dominated by the base term M^π . The safety buffer (second term) grows only logarithmically in the time horizon and is modest in practice. Section ?? provides numerical validation.

Segment-wise generalization: In practice, we use a **segment-wise variant** that groups m types into $L \leq m$ segments (Section 7). This design addresses two concerns: (i) robustness to varying or uncertain type distributions; (ii) for long decode lengths, maintaining separate thresholds for every type may be infeasible. Grouping into segments provides stable scheduling with similar guarantees. Having established theoretical foundations for both known and unknown output lengths, we now validate our algorithms experimentally (Section ??).

6. Numerical Experiments

We evaluate our (Nested) WAIT algorithm through numerical experiments. Our analysis focuses on two distinct scenarios: (i) simulated prompts designed to test the algorithm under controlled conditions, and (ii) a real-world dataset sourced from Zheng et al. (2023) to assess its performance in practical settings.

We conduct all experiments using Microsoft Vidur to simulate an NVIDIA A100 GPU, mirroring the hardware configuration that prior studies used (Agrawal et al. 2024a). We benchmark our WAIT algorithm against vLLM (Kwon et al. 2023) and Sarathi (Agrawal et al. 2023), widely recognized inference engines that employ First-Come-First-Serve (FCFS) scheduling. Specifically, vLLM prioritizes new arrivals, while Sarathi prioritizes ongoing prompts, with both enforcing fixed limits on the total number of tokens and prompts per iteration.

6.1. Experiment on WAIT Algorithm

We first evaluate the WAIT algorithm when prompt output lengths are known upon arrival. We generate simulated prompts with varying input lengths l_j and output lengths l'_j , drawn from distributions that emulate real-world variability. We adjust arrival rates λ_j to simulate both low-traffic

and high-traffic scenarios, testing the algorithm's resilience under stress. Specifically, we generate type j prompts via a Poisson process with rate λ_j and test two scenarios:

- (i) **Low demand:** $m = 2$, $(l_1, l_2) = (10, 10)$, $(l'_1, l'_2) = (10, 20)$, $(\lambda_1, \lambda_2) = (1000, 1000)$;
- (ii) **High demand:** $m = 3$, $(l_1, l_2, l_3) = (20, 20, 20)$, $(l'_1, l'_2, l'_3) = (100, 200, 300)$, $(\lambda_1, \lambda_2, \lambda_3) = (6000, 4000, 2000)$.

For practical implementation, we simplify threshold computation in WAIT. Given a batch size limit B , we set the threshold for type j prompts as $n_j = B \cdot \rho_j / (l'_j + 1)$, where $\rho_j = \lambda_j / \sum_{j'=1}^m \lambda_{j'}$, aligning with Algorithm 1. To ensure a fair comparison, we enforce the same batch size limit across WAIT, Sarathi, and vLLM.

The throughput of WAIT surpasses that of the benchmark algorithms across all tested arrival rates, as illustrated in the left panels of Figure 7 and Figure 8. This improvement is due to WAIT's ability to optimize batch formation, minimizing GPU idle time. Latency, shown in the right panels, remains within acceptable limits, indicating a good balance between efficiency and responsiveness.

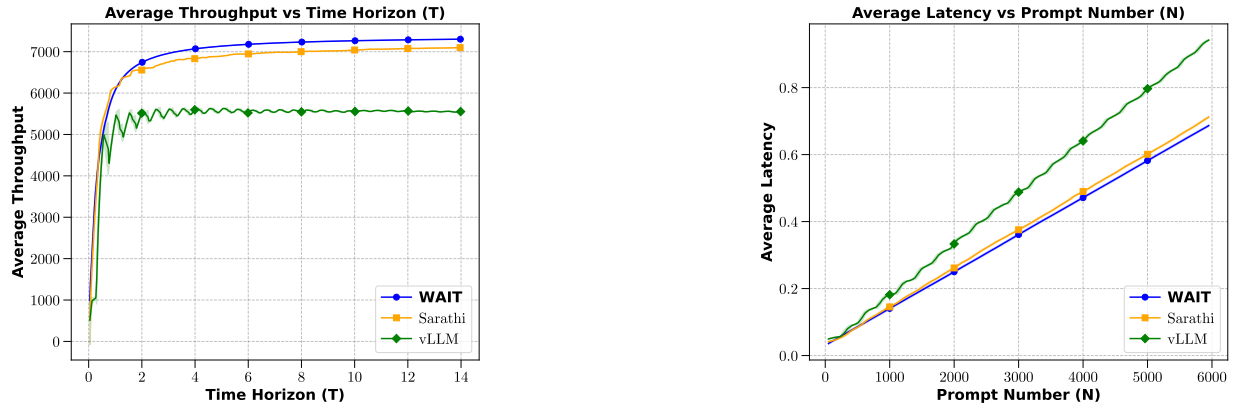


Figure 7 Average throughput and latency across algorithms on the synthetic dataset with low demand.

6.2. Experiment on Nested WAIT Algorithm

Next, we evaluate the Nested WAIT algorithm, designed for scenarios where prompt output lengths are unknown. We test it on both synthetic and real-world datasets, using the same batch size limits across all algorithms and setting thresholds proportional to arrival rates in each segment for practical implementation.

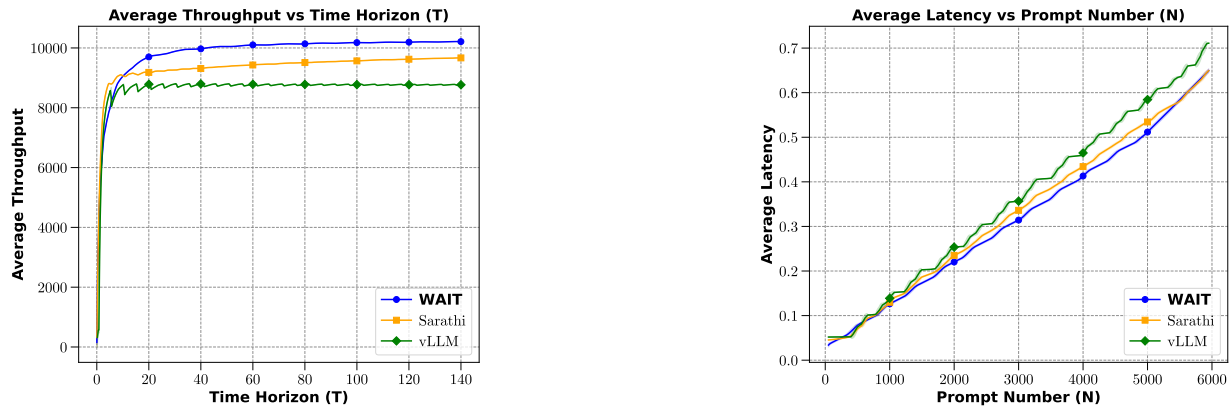


Figure 8 Average throughput and latency across algorithms on the synthetic dataset with high demand.

We begin with a synthetic dataset where $m = 4$, prefill lengths are fixed at 10 tokens, decode lengths are (20, 40, 80, 160), and arrival rates follow a 20 : 40 : 80 : 160 ratio. We set thresholds (n_1, n_2, n_3, n_4) with the ratio 15 : 14 : 12 : 8. Figure 11 shows improved throughput and latency over benchmarks.

For real-world testing, we use the dataset from Zheng et al. (2023), which contains over 210,000 prompts from Vicuna and Chatbot Arena. In this dataset, we use questions as prompts and answers as decoded tokens. Figure 9 shows the distribution of prefill and decode lengths. We randomly sample 50,000 prompts and simulate arrivals via a Poisson process.

To implement Nested WAIT, we group decode lengths (1 to 500 tokens) into 10 bins of 50 tokens each (e.g., [1-50], [51-100], etc). For each bin, we compute the mean prefill length (approximately 60 tokens, as shown in Figure 10) and arrival rates proportional to 23 : 11 : 8 : 7 : 6 : 4 : 3 : 2 : 1 : 1. We set $m = 10$ and thresholds $(n_1, \dots, n_{10})$ with the ratio 66 : 43 : 32 : 24 : 17 : 11 : 7 : 4 : 2 : 1, following (12). Prompts in the k -th decode stage are assigned to segment $\lceil k/50 \rceil$.

We first test a synthetic workload with prefill fixed at 60 tokens and decode lengths clustered into 10 types (50, 100, ..., 500 tokens), with arrival rates matching the real dataset's proportions. We evaluate Nested WAIT with queries per second (QPS) of 55 and 550, shown in Figure 12 and Figure 13, respectively. Nested WAIT outperforms benchmarks in both throughput and latency.

Finally, we test the real dataset with original prefill and decode lengths, using the same thresholds. Figure 14 shows that Nested WAIT achieves higher throughput with a modest increase in latency

(approximately 0.5 seconds per prompt), enhancing system efficiency while maintaining responsiveness. We applied the thresholds, originally designed for static arrival rates, to this time-varying scenario, which resulted in slightly higher latency.

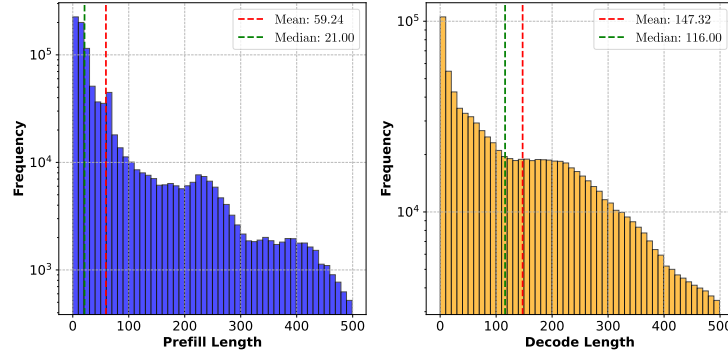


Figure 9 Distribution of prefill and decode lengths in the real dataset.

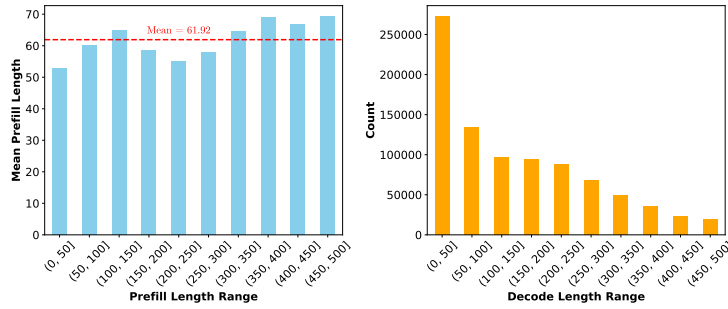


Figure 10 Mean prefill lengths and prompt counts per decode length bin in the real dataset.

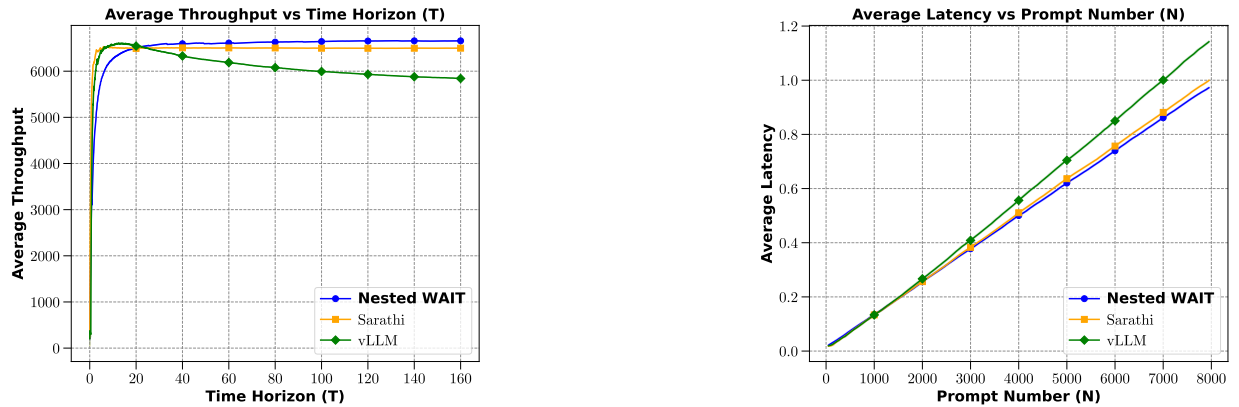


Figure 11 Average throughput and latency across algorithms on the synthetic dataset.

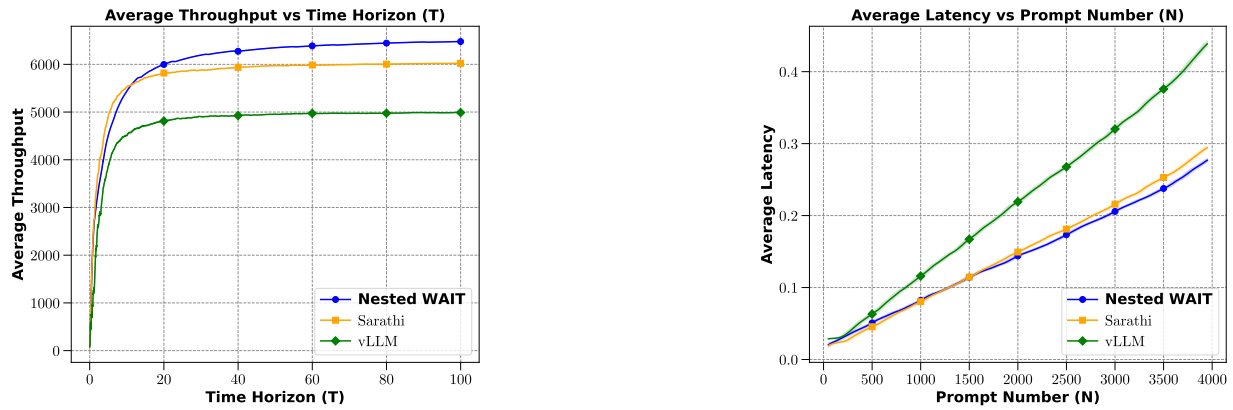


Figure 12 Average throughput and latency across algorithms on the real dataset with clustered output lengths (low arrival rates).

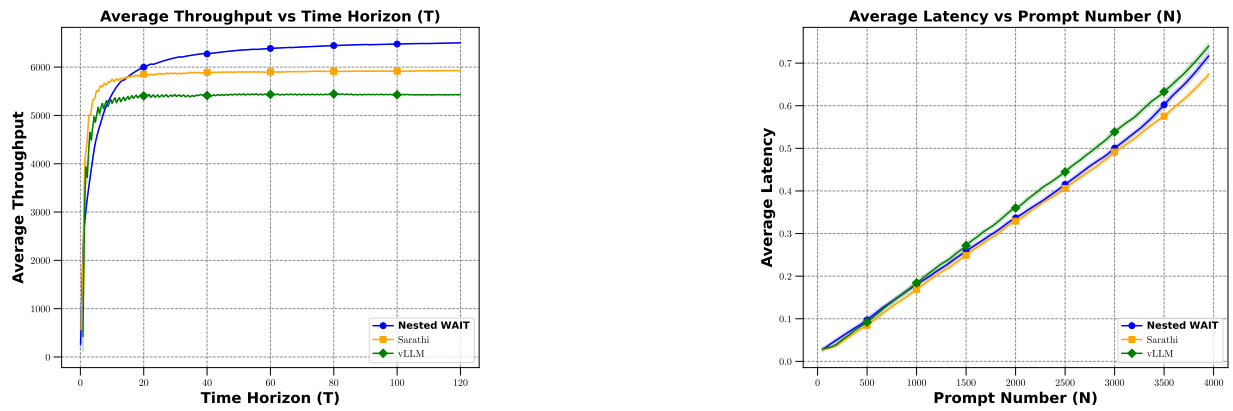


Figure 13 Average throughput and latency across algorithms on the real dataset with clustered output lengths (high arrival rates).

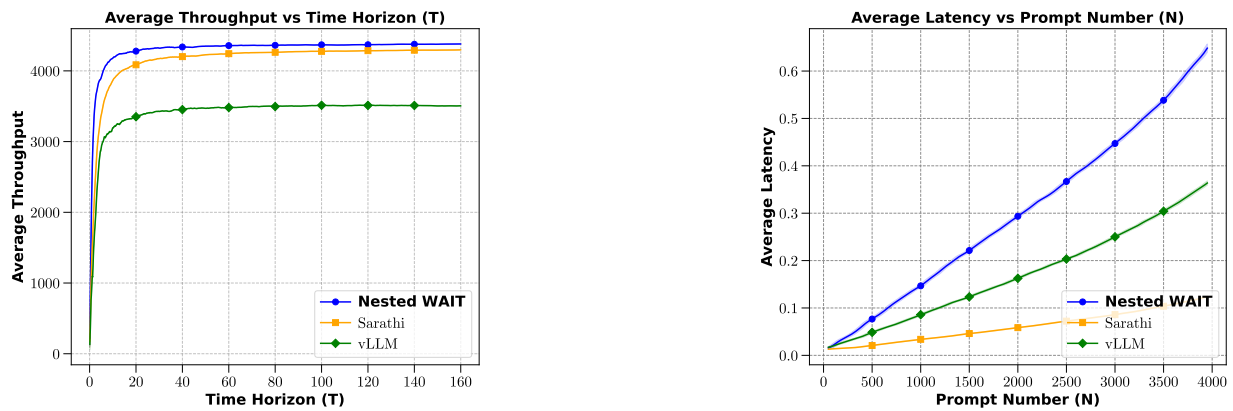


Figure 14 Average throughput and latency across algorithms on the real dataset with actual output lengths.

7. Extensions

The WAIT and Nested WAIT algorithms assume constant arrival rates and a fixed number of segments matching the number of prompt types. In practice, however, LLM inference systems face two additional challenges: (i) **time-varying workloads**, where arrival rates fluctuate throughout the day (e.g., peak hours vs. off-hours); (ii) **numerous prompt types**, where maintaining separate thresholds for hundreds of output lengths becomes infeasible. We address these challenges through two extensions that preserve the core algorithmic principles—eviction prevention and approaching load balance—while adapting to realistic deployment scenarios.

First, we extend Nested WAIT to handle time-varying arrival rates λ_j^t by replacing constant-rate threshold conditions with accumulated arrival constraints over each iteration window. The coupling-based proof technique extends naturally, requiring only that threshold conditions hold at every time instant (Section 7.1). Second, we introduce a **segment-wise design** that groups m prompt types into $L \leq m$ segments, reducing algorithmic complexity while maintaining asymptotic optimality (Section ??). Numerical validation confirms that a moderate segment count (e.g., $L = 10$) suffices in practice, with memory overhead remaining logarithmic.

7.1. Time-Varying Arrival Rates

Time-varying arrival rates capture daily patterns in real LLM services: morning traffic surges, afternoon lulls, evening peaks. The key insight is that our threshold mechanism remains valid as long as the **accumulated arrivals** during each iteration window stay below the completion capacity. Formally, we replace constant rates λ_j with time-dependent rates λ_j^t , assumed continuous and uniformly bounded by $\lambda_{\max} < \infty$ to prevent unbounded bursts.

We analyze the asymptotic optimality of the Nested WAIT algorithm under time-varying arrival rates, denoted λ_j^t for prompt type $j \in [m]$ at time $t \in [0, T]$.

We define the accumulated arrivals of type j over the interval $[t_1, t_2]$ as $\lambda_j[t_1, t_2] := \int_{t_1}^{t_2} \lambda_j^t dt$. Recall from Equation (11) that $\Delta T_{[1,2,\dots,m]}(n_1, \dots, n_m)$ represents the per-iteration time cost when processing exactly n_k prompts at each stage in segment k , for all $k \in [m]$, as in (12), where n_k is the threshold defined in Algorithm 2.

For the Nested WAIT policy defined by $\pi = [n_1, \dots, n_m]$, we consider the following conditions for all $t \in [0, T]$, $0 < \Delta t \leq \Delta T_{[1,2,\dots,m]}(n_1, \dots, n_m)$ and $k = 1, 2, \dots, m-1$, analogous to (12):

$$\begin{aligned} & \sum_{j=1}^m \lambda_j[t, t + \Delta t] \\ & \leq \sum_{j=1}^m \lambda_j[t, t + \Delta T_{[1,2,\dots,m]}(n_1, \dots, n_m)] < n_1, \\ & n_{k+1}/n_k > p_k[t, t + \Delta t], \end{aligned} \tag{15}$$

where $p_k[t, t + \Delta t] = (\sum_{j=k+1}^m \lambda_j[t, t + \Delta t]) / (\sum_{j=k}^m \lambda_j[t, t + \Delta t])$.

We define the total number of iterations over time horizon $[0, T]$ as B . Therefore, we have $T \geq \sum_{b=1}^B \Delta T_{b\text{-th Batch}} \geq B \cdot \min \Delta T_{\text{Batch}}$, where $\Delta T_{b\text{-th Batch}}$ denotes the time cost of the b -th iteration and $\Delta T_{\min} = \min_b \{\Delta T_{\text{Batch}}\}$. The following theorem extends Theorem 2 to time-varying arrival rates.

THEOREM 3. *Suppose the thresholds $\pi = [n_1, \dots, n_m]$ satisfy (15), and the memory $M^{(\zeta, \pi)}$ fulfills:*

$$\begin{aligned} M^{(\zeta, \pi)} & \geq M^\pi + \sum_{k=2}^m (l + l'_{k-1}) \left(n_k + \theta_k^{-1} \ln \left(\frac{m\zeta B}{\delta} \right) \right) \\ & = O \left(2M^\pi + \sum_{k=1}^m \theta_k^{-1} (l + l'_{k-1}) \ln \left(\frac{m\zeta B}{\delta} \right) \right), \end{aligned}$$

where the memory usage M^π is defined as (13) and θ_k is given in (21) with the lower bound in Equation (22). The following asymptotic bounds hold:

$$\begin{aligned} \text{Throughput}^* - \mathbb{E} [\text{Throughput}^{(\zeta, \pi)}] & = O \left((\zeta T)^{-1} \right), \\ \mathbb{E} [\text{Latency}^{(\zeta, \pi)}], \mathbb{E} [\text{TFT}^{(\zeta, \pi)}] & = O(1). \end{aligned}$$

Additionally, the memory usage remains below $M^{\zeta, \pi}$ with probability at least $1 - \delta$ over the time horizon $[0, T]$.

The extension to time-varying rates preserves the dimensionality reduction structure from Theorem 2. The key adaptation is that threshold conditions (15) must hold at every time instant $t \in [0, T]$, replacing constant rates λ_j with accumulated arrivals $\lambda_j[t, t + \Delta t]$ over each iteration window. Because arrival rates are continuous and bounded, the coupling construction remains valid: prompts transitioning from segment k to $k + 1$ still couple to a Poisson process, now with time-varying rate $\sum_{j=k+1}^m \lambda_j^t$. The Lindley recursion and Doob's maximal inequality apply identically, bounding queue lengths with the same logarithmic overhead. Appendix D provides the detailed proof.

7.2. Segment Design

When LLM systems handle hundreds of distinct output lengths (e.g., $m = 500$ in the LMSYS dataset [Zheng et al. \(2023\)](#)), maintaining individual thresholds for each type becomes impractical: the algorithm must track queue lengths separately for every output length, increasing memory overhead and implementation complexity. The **segment-wise design** addresses this by grouping similar output lengths into $L \leq m$ segments, where each segment k aggregates types with comparable decode lengths. This reduces algorithmic complexity from $O(m)$ to $O(L)$ while preserving asymptotic optimality—prompts still classify themselves on-the-fly, but now at segment boundaries rather than individual type boundaries.

We assume constant arrival rates for clarity, though the approach extends to time-varying rates as discussed in Section 7.1. Consider m prompt types with decode lengths $l'_1 < \dots < l'_m$ and arrival rates $\lambda_1, \lambda_2, \dots, \lambda_m$. Our objective is to merge these into L segments, where $L \leq m$. For notational convenience, let $\Delta L := m/L$. We partition the m types into L segments as follows: types 1 to ΔL , types $\Delta L + 1$ to $2\Delta L$, $\dots$, types $(L-1)\Delta L + 1$ to m .

For each segment $k \in \{1, \dots, L\}$, the total arrival rate is defined as $\lambda'_k = \sum_{(k-1)\Delta L + 1 \leq j \leq k\Delta L} \lambda_j$. Additionally, we define $\lambda'(k_1 \rightarrow k_2) := \sum_{r=k_1}^{k_2} \lambda'_r$ for $1 \leq k_1 \leq k_2 \leq L$.

Given a policy $\pi = [n_1, \dots, n_L]$ with threshold n_k for the k -th segment in Algorithm 2, we formulate a linear system analogous to Equation (12):

$$\begin{aligned} \Delta T_{[1:L]}(n_{1:L}) &\leq \frac{n_1}{\sum_{r=1}^L \lambda'_r}, \\ \frac{n_{k+1}}{n_k} &> p_k, \quad \forall k < L, \end{aligned} \tag{16}$$

where $p_k = \frac{\lambda'(k+1 \rightarrow L)}{\lambda'(k \rightarrow L)}$, $\Delta T_{[1,\dots,L]}(n_1, \dots, n_L) = d_0 + d_1 M^\pi(n_1, \dots, n_L)$, and the memory M^π running with n_k at each stage in segment k is given by:

$$M^\pi(n_1, \dots, n_L) = \sum_{k=1}^L n_k \left(l + \frac{k}{2} \Delta L \right) \Delta L.$$

As in Section 7.1, we define the total number of iterations over time horizon $[0, T]$ as B , satisfying $T \geq \sum_{b=1}^B \Delta T_{b\text{-th Batch}} \geq B \cdot \min \Delta T_{\text{Batch}}$.

The segment-wise design achieves two practical goals: (i) **robustness** to varying or uncertain type distributions—if actual output lengths differ slightly from predictions, grouping into segments

absorbs the variation; (ii) **simplicity** in deployment—maintaining $L = 10$ segments is far easier than tracking $m = 500$ individual types. The theoretical guarantee shows that this simplification incurs no asymptotic cost: throughput remains $O(1/(\zeta T))$ away from optimal, with the same logarithmic safety buffer.

THEOREM 4. *For any fixed number of segments L and thresholds $\pi = [n_1, \dots, n_L]$ satisfying Equation (16), if the memory capacity $M^{(\zeta, \pi)}$ satisfies:*

$$\begin{aligned} M^{(\zeta, \pi)} &\geq M^\pi + \sum_{k=2}^L (l + l'_{k-1}) n_k \\ &\quad + \sum_{k=2}^L (l + l'_{k-1}) \theta_k^{-1} \ln \left(\frac{m \zeta B}{\delta} \right) \\ &= O \left(2M^\pi + \sum_{k=1}^L (l + l'_{k-1}) \theta_k^{-1} \ln \left(\frac{m \zeta B}{\delta} \right) \right), \end{aligned} \tag{17}$$

where θ_k is given by Equation (20) with lower bound $8(n_k - n_{k-1} p_k)/n_k$. The performance of Nested WAIT satisfies:

$$\text{Throughput}^* - \mathbb{E} [\text{Throughput}^{(\zeta, \pi)}] = O \left((\zeta T)^{-1} \right),$$

$$\mathbb{E} [\text{Latency}^{(\zeta, \pi)}] = O(1),$$

$$\mathbb{E} [\text{TTFT}^{(\zeta, \pi)}] = O(1).$$

Moreover, memory is not exceeded with probability at least $1 - \delta$ over the total batch counts $b \in \{1, \dots, B\}$ as well as the time horizon $t \in [0, T]$.

Clustering m types into L segments preserves the core threshold structure from Theorem 2. The key observation is that aggregation commutes with the coupling argument: individual arrival rates $(\lambda_1, \dots, \lambda_m)$ are replaced by aggregated rates $(\lambda'_1, \dots, \lambda'_L)$ where $\lambda'_k = \sum_{j \in \text{segment } k} \lambda_j$, but the delayed arrival process for segment k still couples to a Poisson process with rate $\sum_{r=k+1}^L \lambda'_r$. The Lindley recursion and Doob's maximal inequality apply identically at the segment level, yielding the same logarithmic overhead in Equation (17). The proof follows the structure of Theorem 2 in Appendix C.

We validate the segment-wise design numerically using the LMSYS dataset Zheng et al. (2023) (Section 6) with $m = 500$ prompt types, decode lengths from 1 to 500, and fixed prefill length 62.

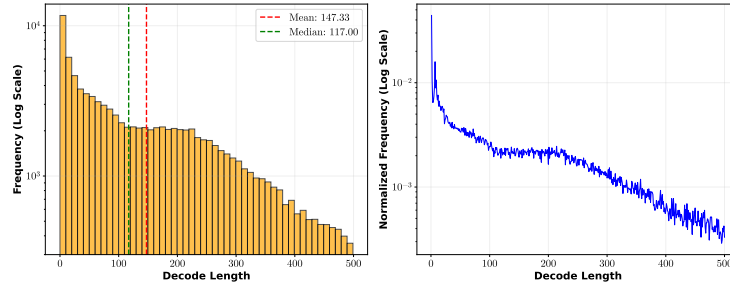


Figure 15 Arrival rate construction from real data, with rates proportional to normalized frequencies.

The distribution of arrival rates, shown in Figure 15, is proportional to normalized frequencies from real user traffic.

We analyze the three components of the memory bound in Equation (17): (1) the peak batch memory usage $\sum_{k=1}^L n_k \left(l + \frac{k}{2} \Delta\right) \Delta$, (2) the queue length bound $\sum_{k=2}^L n_k (l + l'_{k-1})$, and (3) the high-probability queue length bound $\sum_{k=2}^L \theta_k^{-1} (l + l'_{k-1}) \ln \left(\frac{(L-1)(T+1)}{\delta} \right)$.

Figures 16, 17, and 18 plot these three memory components (y-axis) as functions of the number of segments L (x-axis), under varying arrival rates, time horizons T , and confidence levels δ . Each panel shows: Term 1 (blue circles) representing the fluid equilibrium memory from batching, Term 2 (green squares) representing the deterministic queue bound, Term 3 (red triangles) representing the stochastic safety buffer with logarithmic dependence on T and δ , and their sum (black crosses) representing the total memory requirement $M^{(\zeta, \pi)}$. Three key observations emerge: (i) Term 1 (fluid equilibrium memory) dominates Terms 2 and 3 across all parameter settings, confirming that the safety buffer remains modest relative to the base batching requirement. (ii) The total memory (black line) follows a **U-shaped curve** with respect to L : too few segments waste memory by over-provisioning for heterogeneous types (high Term 1 at small L), while too many segments increase overhead from maintaining separate queues. The minimum occurs around $L = 5$ -10, validating the choice of $L = 10$ in Section 6. (iii) Term 3 exhibits smooth growth as δ decreases (comparing left vs. right panels in Figures 16 and 17) and as T increases (comparing Figure 18), consistent with the logarithmic bound in the theoretical analysis.

These extensions demonstrate that the core threshold-based design—eviction prevention through controlled admission—remains robust to realistic deployment conditions. Time-varying arrival rates and segment-wise grouping introduce no fundamental barrier; the coupling-based analysis extends naturally with logarithmic overhead. Section ?? validates these theoretical guarantees through

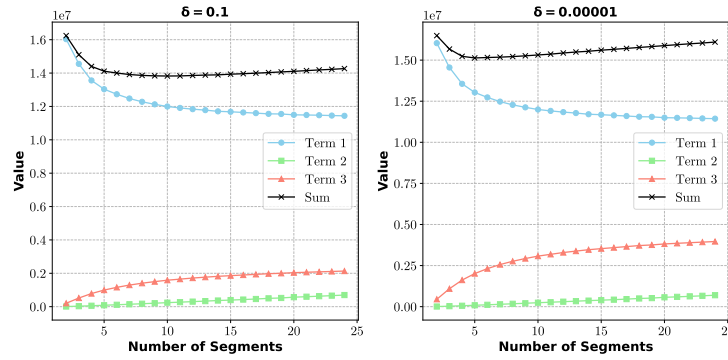


Figure 16 Memory usage proportions under low arrival rate (total rate = 50, $T = 200$). Left: $\delta = 0.1$; Right: $\delta = 10^{-5}$.

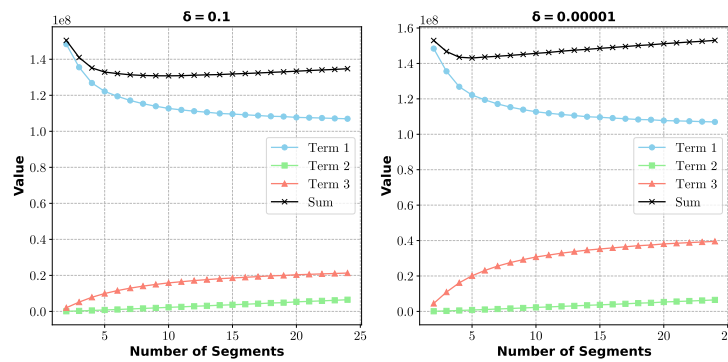


Figure 17 Memory usage proportions under high arrival rate (total rate = 500, $T = 200$). Left: $\delta = 0.1$; Right: $\delta = 10^{-5}$.

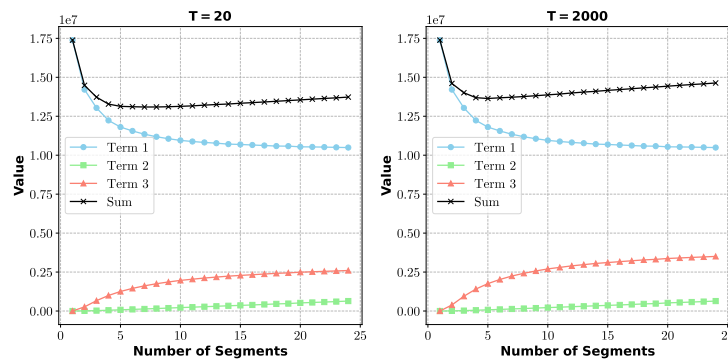


Figure 18 Memory usage proportions with varying time horizons (total rate = 50, $\delta = 0.001$). Left: $T = 20$; Right: $T = 2000$.

experiments on real workloads, showing that Nested WAIT with $L = 10$ segments matches the performance of the full m -type algorithm while significantly reducing implementation complexity.

8. Conclusion, Limitations, and Future Directions

This paper addresses the online scheduling problem for LLM inference under memory constraints, where KV caches grow dynamically during decode and eviction risks complicate traditional scheduling approaches. We introduce the (Nested) WAIT algorithm, which provides theoretical guarantees for throughput and latency in the asymptotic regime, even when output lengths are unknown upon arrival. Our threshold-based admission control prevents eviction cascades while approaching fluid equilibrium. While our algorithms demonstrate strong performance, determining optimal threshold parameters for specific deployment scenarios remains an open problem. Furthermore, extending the algorithm to multi-GPU scenarios—prevalent in real-world applications—introduces additional complexities. Additionally, as system-level optimizations for LLM inference advance with new techniques (e.g., multi-head latent attention), developing analytical frameworks that incorporate these advances remains an important challenge. We discuss several promising directions for future work.

8.1. More Volatile Arrivals

In practical settings, the distribution of incoming prompts becomes non-stationary, violating the stationarity assumption underlying our model. While our algorithm can be directly adapted to scenarios with moderate variations in arrival rates—provided they remain within system capacity—by re-solving the fluid equilibrium, a thorough analysis of its performance in more volatile environments, such as during demand surges, remains an open question. Investigating how the (Nested) WAIT algorithm behaves under such conditions warrants further study.

8.2. Extension to Multi-GPU Systems

Our current research is limited to single-GPU systems, capable of supporting LLMs such as Llama-13B. Extending the algorithm to multi-GPU environments introduces additional considerations, including communication costs, parallelization strategies, and hardware constraints. For example, while single-GPU deployment is relatively straightforward, multi-GPU setups necessitate careful management of data parallelism, tensor parallelism, and pipeline parallelism. Moreover, communication overhead becomes a critical factor, particularly since many large-scale models require multiple GPUs for deployment. Developing a theoretical framework that incorporates these elements requires addressing communication overhead and synchronization constraints specific to distributed GPU architectures.

8.3. Analytical Framework for Up-to-Date LLM Inference Models

In recent years, system-level designs for LLM inference have influenced the field. For instance, the multi-head latent attention (MLA) mechanism introduced by [DeepSeek-AI \(2024\)](#) has achieved a 90% reduction in memory cost through KV cache compression. Integrating these state-of-the-art system-level optimizations into a theoretical framework presents a promising direction for future research.

Endnotes

References

- Agrawal A, Kedia N, Mohan J, Panwar A, Kwatra N, Gulavani B, Ramjee R, Tumanov A (2024a) Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6:351–366.
- Agrawal A, Kedia N, Panwar A, Mohan J, Kwatra N, Gulavani B, Tumanov A, Ramjee R (2024b) Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 117–134.
- Agrawal A, Panwar A, Mohan J, Kwatra N, Gulavani BS, Ramjee R (2023) Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*.
- Aminabadi RY, Rajbhandari S, Awan AA, Li C, Li D, Zheng E, Ruwase O, Smith S, Zhang M, Rasley J, He Y (2022) Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale. *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–15 (IEEE).
- Anthropic (2023) Claude. <https://claude.ai>.
- Anthropic (2025) Claude 3.7 sonnet. URL <https://www.anthropic.com/claude/sonnet>.
- Asmussen S (2003) *Applied probability and queues*, volume 2 (Springer).
- Bai J, Bai S, Chu Y, Cui Z, Dang K, Deng X, Fan Y, Ge W, Han Y, Huang F, et al. (2023) Qwen technical report. *arXiv preprint arXiv:2309.16609*.
- Balkanski E, Rubinstein A, Singer Y (2016) The power of optimization from samples. *Advances in Neural Information Processing Systems* 29.
- Bäuerle N (2002) Optimal control of queueing networks: An approach via fluid models. *Advances in Applied Probability* 34(2):313–328.
- Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, et al. (2020) Language models are few-shot learners. *Advances in neural information processing systems* 33:1877–1901.

- Chen Z, Ye Y, Zhou Z (2025) Adaptively robust llm inference optimization under prediction uncertainty. *arXiv preprint arXiv:2508.14544* .
- Cole R, Roughgarden T (2014) The sample complexity of revenue maximization. *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, 243–252.
- DeepSeek-AI (2024) Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. URL <https://arxiv.org/abs/2405.04434>.
- DeepSeek-AI (2025) Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. URL <https://arxiv.org/abs/2501.12948>.
- Desislavov R, Martínez-Fernández S, Franch X (2021) Compute and energy consumption trends in deep learning inference. *arXiv preprint arXiv:2109.02132* .
- Devanur NR, Hayes TP (2009) The adwords problem: online keyword matching with budgeted bidders under random permutations. *Proceedings of the 10th ACM conference on Electronic commerce*, 71–78.
- Devlin J, Chang MW, Lee K, Toutanova K (2019) Bert: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 4171–4186.
- Durrett R (2019) *Probability: theory and examples*, volume 49 (Cambridge university press).
- Fu Y, Zhu S, Su R, Qiao A, Stoica I, Zhang H (2025) Efficient llm scheduling by learning to rank. *Advances in Neural Information Processing Systems* 37:59006–59029.
- García-Martín E, Rodrigues CF, Riley G, Grahm H (2019) Estimation of energy consumption in machine learning. *Journal of Parallel and Distributed Computing* 134:75–88.
- GitHub (2022) Copilot. URL <https://github.com/features/copilot>.
- Google (2023) Bard. <https://bard.google.com>.
- Hooper C, Kim S, Mohammadzadeh H, Mahoney MW, Shao S, Keutzer K, Gholami A (2025) Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems* 37:1270–1303.
- Im S, Moseley B (2013) Online batch scheduling for flow objectives. *Proceedings of the twenty-fifth annual ACM symposium on Parallelism in algorithms and architectures*, 102–104.
- Jaillet P, Jiang J, Mellou K, Molinaro M, Podimata C, Zhou Z (2025) Online scheduling for llm inference with kv cache constraints. URL <https://arxiv.org/abs/2502.07115>.
- Kang H, Zhang Q, Kundu S, Jeong G, Liu Z, Krishna T, Zhao T (2024) Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm. *arXiv e-prints arXiv:2403*.
- Kaplan J, McCandlish S, Henighan T, Brown TB, Chess B, Child R, Gray S, Radford A, Wu J, Amodei D (2020) Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* .

- Kingman JFC (1962) On queues in heavy traffic. *Journal of the Royal Statistical Society: Series B (Methodological)* 24(2):383–392.
- Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, Gonzalez J, Zhang H, Stoica I (2023) Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626.
- Lattanzi S, Lavastida T, Moseley B, Vassilvitskii S (2020) Online scheduling via learned weights. *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1859–1877 (SIAM).
- Li R, Pal S, Pullu VN, Ryoo JH, John LK, Yadwadkar NJ (2025a) Oneiros: Kv cache optimization through parameter remapping for multi-tenant llm serving. *arXiv preprint arXiv:2507.11507* To appear in ACM SoCC 2025.
- Li W, Wang L, Chai X, Yuan H (2020) Online batch scheduling of simple linear deteriorating jobs with incompatible families. *Mathematics* 8(2):170.
- Li Y, Dai J, Peng T (2025b) Throughput-optimal scheduling algorithms for llm inference and ai agents. URL <https://arxiv.org/abs/2504.07347>, accessed: 2025-04-11.
- Liu P, Lu X (2015) Online unbounded batch scheduling on parallel machines with delivery times. *Journal of Combinatorial Optimization* 29:228–236.
- Liu Y, Whitt W (2011) A network of time-varying many-server fluid queues with customer abandonment. *Operations research* 59(4):835–846.
- Lucier B, Menache I, Naor J, Yaniv J (2013) Efficient online scheduling for deadline-sensitive jobs. *Proceedings of the twenty-fifth annual ACM symposium on Parallelism in algorithms and architectures*, 305–314.
- Maglaras C (2000) Discrete-review policies for scheduling stochastic networks: Trajectory tracking and fluid-scale asymptotic optimality. *The Annals of Applied Probability* 10(3):897–929.
- Mandelbaum A, Massey WA, Reiman MI (1998) Strong approximations for markovian service networks. *Queueing Systems* 30(1):149–201.
- Microsoft (2023) Bing ai. <https://www.bing.com/chat>.
- OpenAI (2024) Gpt-4 technical report. URL <https://arxiv.org/abs/2303.08774>.
- Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, Zhang C, Agarwal S, Slama K, Ray A, et al. (2022) Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35:27730–27744.
- Patel D (2023) Peeling The Onion's Layers: Large Language Models' Cost, Context, and Feature Breakdown. Semianalysis blog, URL <https://semianalysis.com/2023/02/13/peeling-the-onions-layers-large-language/>, accessed: [Insert Date Here].
- Patel P, Choukse E, Zhang C, Shah A, Goiri Í, Maleki S, Bianchini R (2024) Splitwise: Efficient generative llm inference using phase splitting. in 2024 acm/ieee 51st annual international symposium on computer architecture (isca). *IEEE Computer Society, Los Alamitos, CA, USA* 118–132.

- Patterson D, Gonzalez J, Le Q, Liang C, Munguia LM, Rothchild D, So D, Texier M, Dean J (2021) Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350* .
- Pope R, Douglas S, Chowdhery A, Devlin J, Bradbury J, Heek J, Xiao K, Agrawal S, Dean J (2023) Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5:606–624.
- Sheng Y, Zheng L, Yuan B, Li Z, Ryabinin M, Chen B, Liang P, Ré C, Stoica I, Zhang C (2023) Flexgen: High-throughput generative inference of large language models with a single gpu. *International Conference on Machine Learning*, 31094–31116 (PMLR).
- Strait P (1974) On the maximum and minimum of partial sums of random variables. *Pacific Journal of Mathematics* 52(2):585–593.
- Strubell E, Ganesh A, McCallum A (2019) Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243* .
- Tay Y, Dehghani M, Bahri D, Metzler D (2022) Efficient transformers: A survey. *ACM Computing Surveys* 55(6):1–28.
- Touvron H, Lavril T, Izacard G, Martinet X, Lachaux MA, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, et al. (2023) Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* .
- Vee E, Vassilvitskii S, Shanmugasundaram J (2010) Optimal online assignment with forecasts. *Proceedings of the 11th ACM conference on Electronic commerce*, 109–118.
- vLLM Team (2025) vllm v1: A major upgrade to vllm's core architecture. https://docs.vllm.ai/en/latest/getting_started/v1_user_guide.html, blog post, January 27, 2025. "In vLLM V1, the default preemption mode is RECOMPUTE rather than SWAP, as recomputation has lower overhead in the V1 architecture".
- Wang M, Ye Y, Zhou Z (2025) Llm serving optimization with variable prefill and decode lengths. *arXiv preprint arXiv:2508.06133* .
- Wei J, Tay Y, Bommasani R, Raffel C, Zoph B, Borgeaud S, Yogatama D, Bosma M, Zhou D, Metzler D, et al. (2022a) Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682* .
- Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D, et al. (2022b) Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35:24824–24837.
- Wu CJ, Raghavendra R, Gupta U, Acun B, Ardalani N, Maeng K, Chang G, Aga F, Huang J, Bai C, et al. (2022) Sustainable ai: Environmental implications, challenges and opportunities. *Proceedings of Machine Learning and Systems* 4:795–813.
- Yu GI, Jeong JS, Kim GW, Kim S, Chun BG (2022) Orca: A distributed serving system for {Transformer-Based} generative models. *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 521–538.
- Zheng L, Chiang WL, Sheng Y, Li T, Zhuang S, Wu Z, Zhuang Y, Li Z, Lin Z, Xing EP, et al. (2023) Lmsys-chat-1m: A large-scale real-world llm conversation dataset. *arXiv preprint arXiv:2309.11998* .

Zhong Y, Liu S, Chen J, Hu J, Zhu Y, Liu X, Jin X, Zhang H (2024) {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 193–210.

Appendix A: Proofs of Propositions

A.1. Proof of Proposition 1

Proof Outline. We show that when $\lambda_j d_1 (l'_j + 1)(l_j + l'_j/2) \geq 1$, arrivals consistently exceed completions regardless of batch size, causing queue length to grow unboundedly.

Consider a system with only type- j prompts and suppose n_j prompts are in the batch. If equilibrium were to hold, prompts would be uniformly distributed across stages $0, 1, \dots, l'_j$, with $n_j/(l'_j + 1)$ prompts at each stage. The total memory consumption is:

$$M_j = \frac{n_j}{l'_j + 1} \sum_{s=0}^{l'_j} (l_j + s) = n_j \left(l_j + \frac{l'_j}{2} \right).$$

The iteration time to process this batch is:

$$\tau_j = d_0 + d_1 M_j = d_0 + d_1 n_j \left(l_j + \frac{l'_j}{2} \right).$$

During this iteration, the expected number of arriving prompts is:

$$\text{Arrivals}_j = \tau_j \cdot \lambda_j = d_0 \lambda_j + \lambda_j d_1 n_j \left(l_j + \frac{l'_j}{2} \right),$$

while the number of completions is $n_j/(l'_j + 1)$ (prompts at stage l'_j complete). The arrival-completion gap satisfies:

$$\text{Arrivals}_j - \frac{n_j}{l'_j + 1} = d_0 \lambda_j + n_j \left[\lambda_j d_1 \left(l_j + \frac{l'_j}{2} \right) - \frac{1}{l'_j + 1} \right] \geq d_0 \lambda_j > 0,$$

where the inequality follows from $\lambda_j d_1 (l'_j + 1)(l_j + l'_j/2) \geq 1$.

Since this gap is positive for all $n_j > 0$, the queue length after N iterations satisfies:

$$\text{Queue Length}(N) \geq d_0 \lambda_j \cdot N.$$

The latency for a prompt arriving at iteration N grows as $\Omega(N)$, establishing instability.

A.2. Proof of Proposition 2

Proof Outline. The expected throughput of any online policy cannot exceed the fluid throughput **Throughput***, since throughput is bounded by the arrival rate of output tokens.

In the fluid model, the equilibrium parameters are:

$$\Delta T = \frac{d_0}{1 - d_1 \sum_{j=1}^m \lambda_j (l'_j + 1)(l_j + l'_j/2)}, \quad n_j^* = \Delta T \cdot \lambda_j (l'_j + 1),$$

$$M^* = \Delta T \cdot \sum_{j=1}^m \lambda_j (l'_j + 1)(l_j + l'_j/2), \quad \text{Throughput}^* = \sum_{j=1}^m \lambda_j l'_j.$$

In any stochastic system, throughput is counted only after prompts complete. Over time horizon $[0, T]$, let Arrivals_j^T denote the total arrivals of type- j prompts. The total output tokens generated cannot exceed the total output length of all arrivals:

$$\text{Throughput}^{(\pi, T)} \leq \sum_{j=1}^m \text{Arrivals}_j^T \cdot l'_j.$$

Taking expectations and dividing by T :

$$\mathbb{E}[\text{Throughput}^{(\pi, T)}] \leq \sum_{j=1}^m \mathbb{E} \left[\frac{\text{Arrivals}_j^T}{T} \right] \cdot l'_j = \sum_{j=1}^m \lambda_j l'_j = \text{Throughput}^*.$$

A.3. Proof of Proposition 3

Proof Outline. We construct a two-type example where FCFS suffers from cascading evictions even when capacity equals equilibrium memory M^* , leading to $\Omega(1)$ throughput gap.

Consider two prompt types: type 1 with $(l_1, l'_1, \lambda_1) = (1, 1, 1)$ and type 2 with $(l_2, l'_2, \lambda_2) = (1, 2, 1)$.

Fluid Equilibrium. The equilibrium equations $d_0 + d_1(1.5n_1 + 2n_2) = n_1/2 = n_2/3$ yield:

$$K = \frac{d_0}{1 - 9d_1}, \quad n_1^* = 2K, \quad n_2^* = 3K, \quad M^* = 9K.$$

Setting $K = 1$ (without loss of generality), we have $n_1^* = 2, n_2^* = 3, M^* = 9$.

Equilibrium Batch Composition. In equilibrium, the batch contains:

- Type 1: 1 prompt at stage 0 (memory = 1), 1 prompt at stage 1 (memory = 2, completing)
- Type 2: 1 prompt at stage 0 (memory = 1), 1 at stage 1 (memory = 2), 1 at stage 2 (memory = 3, completing)

Overflow Probability. Let (X_1, X_2) denote arrivals during one iteration, where $X_j \sim \text{Poisson}(1)$. The combinations that do not cause memory overflow within two iterations are: $(0, 0), (1, 0), (0, 1), (1, 1), (2, 0)$. All other combinations lead to overflow. The overflow probability is:

$$p = 1 - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{2e^2} = 1 - \frac{9}{2e^2} \approx 0.391.$$

Since overflow probability is bounded away from zero, FCFS experiences eviction with constant probability at each iteration. This leads to queue length growth $\Omega(t)$ and throughput gap $\Omega(1)$.

A.4. Proof of Proposition 5

Proof Outline. Using the same two-type example, we show that when output lengths are unknown until completion, any online policy faces either memory underutilization or constant overflow probability.

Consider the same setup as Proposition 3: type 1 with $(l_1, l'_1, \lambda_1) = (1, 1, 1)$ and type 2 with $(l_2, l'_2, \lambda_2) = (1, 2, 1)$. The fluid equilibrium gives $M^* = 9$.

The key constraint is that prompt types cannot be distinguished until a type-2 prompt enters stage 2 (generates its second output token). At stage 0 and stage 1, both types appear identical to the scheduler.

Consider any admissible batching policy. For each possible batch configuration, one of the following must hold:

1. **Memory underutilization:** The batch leaves significant memory unused, causing expected arrivals to exceed completions during the iteration.
2. **Overflow risk:** The batch fully utilizes memory, but faces constant probability $\rho > 0$ of overflow when arrivals exceed expectations.

Since throughput is counted only upon prompt completion, and the fluid throughput equals expected arrivals' output length per unit time, any policy that either underutilizes memory or experiences overflow incurs a throughput gap of at least $\Omega(\rho)$ compared to **Throughput***.

Appendix B: Proof of Theorem 1

Proof Outline. This proof establishes asymptotic optimality of the WAIT algorithm when output lengths are known. The structure proceeds in two parts:

1. **Single-type case:** We construct a coupled dominating process using the Lindley recursion and apply Kingman's bound for queue length expectations.
2. **Multiple-type case:** We extend to m types by applying the coupling construction type-by-type, leveraging that the fixed threshold n_j decouples inter-type dynamics.

Key techniques include: Lindley recursion for queue length bounds (Asmussen 2003, Proposition 6.3), coupling construction to dominate the original process, Kingman's bound for negative drift processes (Kingman 1962), and standard random walk results (Strait 1974).

Throughout this appendix, we use b to denote the *batch index* (the count of completed batch iterations), which is distinct from the stage index $s \in \{0, 1, \dots, l'_j\}$ used in the main text for decode

stages. We use t for continuous time and T (or B) for the total number of batches over the time horizon.

We first consider the single-type case, then extend to multiple types.

B.0.1. Single-Type Case Consider a single prompt type with threshold n . For general scheduling algorithms, analyzing this system in continuous time is challenging: KV cache grows dynamically during decode, arrivals are stochastic, and eviction risks complicate the dynamics.

The key insight of the WAIT policy is that its *threshold mechanism achieves dimensionality reduction*. By waiting until exactly n prompts accumulate before initiating a batch, the algorithm enforces a fixed batch size, which ensures constant processing time ΔT per batch. This decouples the complex continuous-time dynamics into a tractable discrete-time process. This reduction—from a complex memory-constrained queueing system to a simple random walk analysis—is enabled by the algorithm design, not an inherent simplicity of the problem.

Specifically, let $\tilde{\lambda}$ denote the continuous-time arrival rate. The WAIT policy's fixed batch size implies that arrivals during each batch follow $\text{Poisson}(\tilde{\lambda} \cdot \Delta T)$. Defining $\lambda = \tilde{\lambda} \cdot \Delta T$ (expected arrivals per batch), we can analyze the system as an *embedded Markov chain* observed at batch completion times, indexed by $b \in \mathbb{N}$. Setting $n = \lambda$ corresponds to critical loading. The throughput loss arises from “stuck” iterations where insufficient prompts are available.

LEMMA 1 (Queue Length and Stuck Time). *This lemma connects queue accumulation to throughput loss through stuck iterations. Let X^b denote the number of arrivals during batch b , where $X^b \sim \text{Poisson}(\lambda)$. Let W^b denote the queue length after batch b , with $W^0 = 0$. The state transition is:*

$$W^{b+1} = W^b + X^b - \lambda \cdot \mathbf{1}\{W^b + X^b \geq \lambda\}.$$

Define B_{stuck} as the number of stuck iterations (where $W^b + X^b < \lambda$). Then:

$$\mathbb{E}[W^B] = \lambda \cdot \mathbb{E}[B_{\text{stuck}}].$$

The proof is in Appendix E.1. This lemma connects queue accumulation to throughput loss: stuck iterations directly translate to waiting prompts. To bound $\mathbb{E}[W^B]$, we construct a coupled dominating process that admits tractable analysis via classical queueing techniques.

LEMMA 2 (Coupled Dominating Process). *The coupled process starts with a safety buffer (2λ vs 0) and maintains dominance throughout. Define $\{\tilde{W}^b\}$ by:*

$$\tilde{W}^0 = 2\lambda, \quad \tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\}.$$

Then $\tilde{W}^b \geq W^b + \lambda$ for all $b \in \mathbb{N}$.

The proof is in Appendix E.2.

Lindley recursion. To analyze the coupled process, we apply the Lindley recursion representation (Asmussen 2003, Proposition 6.3):

LEMMA 3 (Lindley Representation). *Let $S^k = \sum_{r=1}^k (X^r - \lambda)$ be the partial sum process. Then:*

$$\tilde{W}^B = 2\lambda + \max(S^B, S^B - S^1, \dots, S^B - S^{B-1}, 0).$$

This classical queueing representation is made applicable by the WAIT policy's threshold design. Define $\xi^k = X^k - \lambda$. We use time-reversal symmetry to simplify the maximum expression:

$$\max(S^B, S^B - S^1, \dots, S^B - S^{B-1}, 0) = \max_{1 \leq k \leq B} \left(\sum_{r=k}^B \xi^r \right)^+ \stackrel{d}{=} \max_{1 \leq k \leq B} \left(\sum_{r=1}^k \eta^r \right)^+, \quad \eta^r \stackrel{d}{=} X^1 - \lambda.$$

Thus $\tilde{W}^B \stackrel{d}{=} 2\lambda + \max_{1 \leq k \leq B} (\sum_{r=1}^k \eta^r)^+$.

Since $\tilde{W}^B \geq W^B + \lambda$, we have $\mathbb{E}[W^B] \leq \mathbb{E}[\tilde{W}^B] - \lambda$. Let $Y^B = \max_{1 \leq k \leq B} (\sum_{r=1}^k \eta^r)^+$. By (Strait 1974, Lemma 2):

LEMMA 4 (Random Walk Maximum).

$$\mathbb{E}[Y^B] = \mathbb{E} \left[\max_{1 \leq k \leq B} (S^k)^+ \right] = \sum_{k=1}^B \frac{1}{k} \mathbb{E}[(S^k)^+].$$

This standard result (Strait 1974) decomposes the maximum into a sum over batch indices. For $\mathbb{E}[(S^k)^+]$, by the Cauchy-Schwarz inequality and $\text{Var}(S^k) = \lambda k$, we have $\mathbb{E}[(S^k)^+] \leq \sqrt{\lambda k}$. Hence:

$$\mathbb{E}[Y^B] \leq \sqrt{\lambda} \sum_{k=1}^B k^{-1/2} \sim \sqrt{\lambda B} \quad \text{as } B \rightarrow \infty.$$

LEMMA 5 (Throughput Gap Bound).

$$\mathbb{E}[W^B] - \lambda \leq \mathbb{E}[Y^B] \leq \sqrt{\lambda} \sum_{k=1}^B \frac{1}{\sqrt{k}} \sim \sqrt{\lambda B}, \quad B \rightarrow \infty.$$

The throughput gap is thus $\frac{1}{B} \mathbb{E}[W^B] = O(B^{-1/2})$.

Asymptotic Regime. In the asymptotic regime where the time horizon scales as $\zeta \rightarrow \infty$:

$$\frac{1}{B^\zeta} \mathbb{E}[W^{B,\zeta}] = \frac{\lambda^\zeta}{B^\zeta} + \frac{(\lambda B)^{\zeta/2}}{B^\zeta} = \left(\frac{\lambda}{B}\right)^\zeta + \lambda^{\zeta/2} B^{-\zeta/2} \rightarrow 0 \quad \text{as } \zeta \rightarrow \infty.$$

B.0.2. Multiple-Type Case We now extend to m prompt types. Consider the WAIT policy $\pi = [n_1, \dots, n_m]$, where n_j is the threshold for type- j prompts.

Policy Class Π . Define the processing time for a full batch (containing all m types):

$$\Delta T_{[1,\dots,m]} = d_0 + d_1 \cdot \sum_{j=1}^m n_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right). \quad (18)$$

Approaching Load Balance. The constraint $\Delta T_{[1,\dots,m]} \leq n_j / \lambda_j$ (Equation 19) embodies the "approaching load balance" principle. For each type j , expected arrivals during a full batch equal $\lambda_j \cdot \Delta T_{[1,\dots,m]}$. The constraint ensures arrivals do not exceed batch size n_j , preventing queue accumulation and eviction risk. The equilibrium policy π^* achieves equality (arrivals = completions), maximizing throughput while maintaining stability.

The policy parameters must satisfy:

$$\begin{aligned} \Delta T_{[1,\dots,m]} &\leq \frac{n_j}{\lambda_j} \text{ for all } j \in [m], \\ M^\pi &= \sum_{j=1}^m n_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right) \geq M^*. \end{aligned} \quad (19)$$

These constraints ensure that for any $\pi \in \Pi$: when all m types are batched together, arrivals do not exceed completions for each type (i.e., $\text{Arrival}_j \leq \text{Completion}_j$). The equilibrium policy corresponds to $\pi^* = [\frac{n_1^*}{l'_1+1}, \dots, \frac{n_m^*}{l'_m+1}]$ when $M^\pi = M^*$.

Notation. Let b denote the batch index, J^b the set of types in batch b , $t_s(b)$ the start time and $t_e(b)$ the end time of batch b . The waiting time $t_s(b+1) - t_e(b) > 0$ occurs only when no type meets its threshold.

State Transition. At batch $b+1$:

$$W_{(j)}^{t_e(b+1)} = W_{(j)}^{t_e(b)} + X_{(j)}^{[t_e(b), t_s(b+1)]} + Y_{(j)}^{[t_s(b+1), t_e(b+1)]} - n_j \cdot \mathbf{1}\{j \in J^{b+1}\}, \quad j = 1, \dots, m,$$

where $X_{(j)}^{[t_e(b), t_s(b+1)]} \sim \text{Poisson}(\lambda_j \cdot (t_s(b+1) - t_e(b)))$ and $Y_{(j)}^{[t_s(b+1), t_e(b+1)]} \sim \text{Poisson}(\lambda_j \cdot (t_e(b+1) - t_s(b+1)))$ are independent arrival counts during waiting and processing periods.

Batch Processing Time. Recall from (18) that $\Delta T_{[1,\dots,m]}$ denotes the processing time for a full batch. Under fixed thresholds, this time is constant across batches.

Type Decoupling. Fixed thresholds n_j enable type decoupling. Each type j has independent arrivals during batch processing: $X_{(j)}^b \sim \text{Poisson}(\lambda_j \cdot \Delta T_{[1, \dots, m]})$. Arrivals depend only on the type-specific rate λ_j and batch time $\Delta T_{[1, \dots, m]}$, not on other types' queue states. This independence allows us to analyze each type separately and sum the resulting bounds.

LEMMA 6 (Multiple-Type Coupled Process). *Define a coupled process $\{\tilde{W}_{(j)}^b\}$ indexed by batch b :*

$$\tilde{W}_{(j)}^0 = 2n_j, \quad \tilde{W}_{(j)}^{b+1} = \max\{2n_j, \tilde{W}_{(j)}^b + X_{(j)}^b - n_j\}, \quad X_{(j)}^b \sim \text{Poisson}(\lambda_j \cdot \Delta T_{[1, \dots, m]}).$$

Then $\tilde{W}_{(j)}^b \geq W_{(j)}^b$ for all $b \geq 0$ and $j = 1, \dots, m$.

The proof is in Appendix E.3. Define $Y_{(j)}^b = X_{(j)}^b - n_j$. The coupled process becomes:

$$\tilde{W}_{(j)}^0 = 2n_j, \quad \tilde{W}_{(j)}^{b+1} = \max\{2n_j, \tilde{W}_{(j)}^b + Y_{(j)}^b\}.$$

Case 1: Zero Drift ($\Delta T_{[1, \dots, m]} = n_j / \lambda_j$). When $\lambda_j \cdot \Delta T_{[1, \dots, m]} = n_j$, we have $\mathbb{E}[Y_{(j)}^b] = 0$ (zero drift). By Lemma 5:

$$\mathbb{E}[W_{(j)}^B] \leq \mathbb{E}[\tilde{W}_{(j)}^B] \leq \lambda_j + \sqrt{\lambda_j B}.$$

Case 2: Negative Drift ($\Delta T_{[1, \dots, m]} < n_j / \lambda_j$). When $\mathbb{E}[Y_{(j)}^b] = \lambda_j \Delta T_{[1, \dots, m]} - n_j < 0$, the coupled process has negative drift. Since

$$\text{Var}(Y_{(j)}^b) = \lambda_j \cdot \Delta T_{[1, \dots, m]}, \quad \left| \mathbb{E}[Y_{(j)}^b] \right| = n_j - \lambda_j \cdot \Delta T_{[1, \dots, m]},$$

by Kingman's bound (Kingman 1962):

$$\mathbb{E}[W_{(j)}^B] \leq \mathbb{E}[\tilde{W}_{(j)}^B] \leq 2n_j + \frac{\lambda_j \cdot \Delta T_{[1, \dots, m]}}{2(n_j - \lambda_j \cdot \Delta T_{[1, \dots, m]})}.$$

Equilibrium Case ($C = M^*$). When $\pi^* = [\frac{n_1^*}{l_1^*+1}, \dots, \frac{n_m^*}{l_m^*+1}]$, all types satisfy $\Delta T_{[1, \dots, m]} = n_j / \lambda_j$ (zero drift). Summing over all types:

$$\frac{1}{B} \sum_{j=1}^m \mathbb{E}[W_{(j)}^B] \leq \frac{1}{B} \sum_{j=1}^m \lambda_j + \sum_{j=1}^m \sqrt{\frac{\lambda_j}{B}} = O(B^{-1/2}).$$

Asymptotic Regime. As $\zeta \rightarrow \infty$:

$$\frac{1}{\zeta B} \sum_{j=1}^m \lambda_j + \sum_{j=1}^m \sqrt{\frac{\lambda_j}{\zeta B}} \rightarrow 0.$$

The expected end-to-end latency scales as $O(B^{-1/2})$.

Strictly Negative Drift Case. When $\Delta T_{[1,\dots,m]} < n_j/\lambda_j$ for all j , the throughput gap is $O(B^{-1})$:

$$\frac{1}{B} \sum_{j=1}^m \mathbb{E} \left[W_{(j)}^B \right] \leq \frac{1}{B} \sum_{j=1}^m \left(2n_j + \frac{\lambda_j \cdot \Delta T_{[1,\dots,m]}}{2(n_j - \lambda_j \cdot \Delta T_{[1,\dots,m]})} \right) = O(B^{-1}).$$

Appendix C: Proof of Theorem 2

Proof Outline. This proof establishes asymptotic optimality of Nested WAIT when output lengths are unknown. The algorithm achieves *on-the-fly classification*: prompts classify themselves by their actual output lengths at segment boundaries, requiring no prediction. The structure proceeds as follows:

1. **First segment analysis:** All prompts enter segment 1 with Poisson arrivals. We couple with a Lindley process and apply Kingman's inequality for bounded expectations.
2. **Subsequent segments analysis:** After completing segment $k - 1$, prompts naturally reveal whether they continue to segment k . This type revelation follows Binomial thinning: a fraction p_k of completions continue, while short prompts finish without being delayed by longer ones. We apply Lindley coupling to each segment.
3. **High-probability bounds prevent eviction:** Expected bounds ensure average stability, but stochastic fluctuations risk overflow. We construct an exponential martingale and apply Doob's inequality to obtain high-probability memory bounds, preventing eviction cascades.

We adopt notation consistent with the main text (Section 5). Let l denote the common input length (tokens after prefill), and let l'_j denote the output length for type- j prompts. We use b to denote the *batch index* (count of completed iterations) and $k \in \{1, \dots, m\}$ to denote the *segment index*, where segment k contains prompts with output lengths in $(l'_{k-1}, l'_k]$. The threshold for segment k is n_k (from Equation (12)). We denote by $W_{(k)}^b$ the queue length for segment k after batch b , and by B the total number of batches over the time horizon. The thinning probability $p_k = (\sum_{j=k}^m \lambda_j) / (\sum_{j=k-1}^m \lambda_j)$ represents the fraction of prompts completing segment $k - 1$ that continue to segment k .

C.1. First Segment Analysis

All prompts initially enter segment 1 with Poisson arrivals at aggregate rate $\sum_{j=1}^m \lambda_j$. We construct a dominating process and apply Kingman's inequality to bound expected queue length.

For the first segment ($k = 1$), the state transition is:

$$W_{(1)}^{b+1} = W_{(1)}^b + Y_{(1)}^b - n_1 \cdot \mathbf{1}\{W_{(1)}^b + Y_{(1)}^b \geq n_1\}, \quad Y_{(1)}^b \sim \text{Poisson}(\Delta T_b \cdot \sum_{j=1}^m \lambda_j),$$

where ΔT_b denotes the processing time of batch b . Since $\Delta T_b \leq \Delta T_{[1, \dots, m]}$ (the time for a full batch), we construct a dominating process:

$$\bar{W}_{(1)}^{b+1} = \bar{W}_{(1)}^b + \bar{Y}_{(1)}^b - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^b + \bar{Y}_{(1)}^b \geq n_1\}, \quad \bar{Y}_{(1)}^b \sim \text{Poisson}(\Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j).$$

This is further dominated by a Lindley process:

$$\tilde{W}_{(1)}^0 = 2n_1, \quad \tilde{W}_{(1)}^{b+1} = \max\{2n_1, \tilde{W}_{(1)}^b + \bar{Y}_{(1)}^b - n_1\}.$$

Since $\Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j < n_1$ (negative drift), by Kingman's inequality (Kingman 1962):

$$\mathbb{E} \left[W_{(1)}^b \right] \leq 2n_1 + \frac{\Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j}{2 \left(n_1 - \Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j \right)}.$$

Remark. Prompts waiting in the first segment are stored in CPU and do not consume GPU memory.

C.2. Subsequent Segments Analysis ($k \geq 2$)

For segment k ($2 \leq k \leq m$), the arrival process reflects on-the-fly classification: prompts that completed segment $k-1$ reveal whether their output lengths exceed l'_{k-1} . This type revelation follows binomial thinning with probability p_k .

Arrivals come from prompts that completed segment $k-1$:

$$Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k), \quad \text{where } p_k = \frac{\sum_{j=k}^m \lambda_j}{\sum_{j=k-1}^m \lambda_j}.$$

On-the-fly Classification via Binomial Thinning. The arrival process $Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k)$ reflects the *on-the-fly classification mechanism*, not merely a modeling choice. After segment $k-1$ processing, exactly n_{k-1} prompts complete that stage. Among these, a fraction $p_k = \frac{\sum_{j=k}^m \lambda_j}{\sum_{j=k-1}^m \lambda_j}$ have output lengths exceeding l'_{k-1} . These prompts naturally continue to segment k . Prompts classify themselves by their actual output lengths—no prediction needed. Short prompts complete quickly without being delayed by longer ones.

The state transition is:

$$W_{(k)}^{b+1} = W_{(k)}^b + Y_{(k)}^b - n_k \cdot \mathbf{1}\{W_{(k)}^b + Y_{(k)}^b \geq n_k\},$$

where b indexes the batches containing segment $k-1$. Note that each segment has its own batch index, but all are bounded by the total batch count B .

Define $X_{(k)}^b = Y_{(k)}^b - n_k$. We construct a coupled dominating process:

LEMMA 7 (Coupled Process for Segment k). Define $\{\tilde{W}_{(k)}^b\}$ by:

$$\begin{aligned}\tilde{W}_{(k)}^0 &= n_k, \\ \tilde{W}_{(k)}^{b+1} &= \max\{n_k, \tilde{W}_{(k)}^b + X_{(k)}^b\}.\end{aligned}$$

Then $\tilde{W}_{(k)}^b \geq W_{(k)}^b$ for all $b \in \mathbb{N}$.

The proof is in Appendix E.4. The process $\{\tilde{W}_{(k)}^b\}$ has the Lindley representation:

$$\begin{aligned}\tilde{W}_{(k)}^{b+1} &= n_k + \max_{0 \leq i \leq b} \{S_{(k)}^i\}, \\ \text{where } S_{(k)}^i &= \sum_{r=1}^i X_{(k)}^r, \quad S_{(k)}^0 = 0.\end{aligned}$$

Expected Queue Length. Since $\mathbb{E}[X_{(k)}^b] = n_{k-1}p_k - n_k < 0$ (negative drift), by Kingman's inequality (Kingman 1962):

$$\mathbb{E}[W_{(k)}^b] \leq \mathbb{E}[\tilde{W}_{(k)}^b] \leq n_k + \frac{n_{k-1} \cdot p_k (1 - p_k)}{2(n_k - n_{k-1} \cdot p_k)}.$$

Negative drift ensures queue stability, preventing unbounded growth that would lead to eviction. This bound ensures finite expected queue length for each segment $k \geq 2$, controlling expected GPU memory consumption. The segment structure ensures shorter prompts complete without waiting for longer ones to finish.

C.3. Throughput Gap Bound

Using B as the total batch count (upper bound for each segment's batch index), the throughput gap is:

$$\frac{1}{B} \sum_{k=1}^m \mathbb{E}[W_{(k)}^B] \leq \frac{2n_1}{B} + \frac{\Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j}{2B(n_1 - \Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j)} + \frac{1}{B} \sum_{k=2}^m n_k + \frac{1}{B} \sum_{k=2}^m \frac{n_{k-1} \cdot p_k (1 - p_k)}{2(n_k - n_{k-1} \cdot p_k)}.$$

Asymptotic Regime. As $\zeta \rightarrow \infty$, the throughput gap is $O((\zeta B)^{-1})$. The expected latency and TTFT are $O(1)$.

C.4. High-Probability Memory Bound

High-Probability Bounds Prevent Eviction. Expected queue length bounds ensure average stability. However, stochastic fluctuations may cause $W_{(k)}^b$ to exceed these limits, risking memory overflow and eviction. We derive high-probability bounds to quantify this overflow risk. By choosing capacity C to ensure overflow probability $< \delta$ across all batches and segments, we prevent eviction cascades.

Martingale Construction. To obtain high-probability bounds on queue length, we need to control tail probabilities of the random walk maximum $\max_{0 \leq i \leq s-1} \{S_{(k)}^i\}$. Expected bounds alone cannot prevent rare but catastrophic overflow events. We construct an exponential martingale that enables Doob's inequality, converting expected bounds into probabilistic guarantees.

We need to estimate

$$P\left(\max\{0, S_{(k)}^1, \dots, S_{(k)}^{s-1}\} \geq c\right) = P\left(\max_{0 \leq i \leq s-1} \{S_{(k)}^i\} \geq c\right), \forall c > 0, \text{ where } S_{(k)}^0 = 0.$$

Define the exponential martingale $M_{(k)}^i = e^{\theta S_{(k)}^i}$. For this to be a martingale, the moment generating function must equal 1, ensuring zero expected drift in log space:

$$\phi_{(k)}(\theta) = \mathbb{E}\left[e^{\theta X_{(k)}^i}\right] = e^{-\theta n_k}(1 - p_k + p_k e^\theta)^{n_{k-1}} = 1.$$

The following lemma establishes existence and properties of the solution $\theta_k > 0$ that satisfies this martingale condition (proof in Appendix E.5).

LEMMA 8. *Suppose $n_{k-1} > n_k > n_{k-1}p_k$ and $p_k \in (0, 1)$. Then there exists a unique solution $\theta_k > 0$ to the equation*

$$e^{-\theta_k n_k}(1 - p_k + p_k e^{\theta_k})^{n_{k-1}} = 1, \quad (20)$$

where θ_k is strictly increasing with respect to $D = n_k - n_{k-1}p_k$ and satisfies the lower bound

$$\theta_k \geq \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}.$$

When $\theta_k > 0$ satisfies $\phi(\theta_k) = 1$, the process $M_{(k)}^i$ is a martingale starting at $M_{(k)}^0 = e^{\theta_k \cdot 0} = 1$. This zero-drift property in log space enables Doob's inequality to bound tail probabilities.

By Doob's martingale inequality (Durrett 2019), we have that

$$P\left(\max_{1 \leq i \leq b-1} \{e^{\theta_k S_{(k)}^i}\} \geq a\right) \leq \frac{\mathbb{E}[M^i]}{a} = \frac{\mathbb{E}[M^0]}{a} = \frac{1}{a}.$$

Exponentiating both sides of the inequality event, we obtain:

$$P\left(\max_{1 \leq i \leq b-1} \{e^{\theta_k S_{(k)}^i}\} \geq e^{\theta_k c}\right) \leq \frac{1}{e^{\theta_k c}} \Rightarrow P\left(\max_{1 \leq i \leq b-1} \{S_{(k)}^i\} \geq c\right) \leq e^{-\theta_k c}.$$

Using the dominance $\tilde{W}_{(k)}^b = n_k + \max_{0 \leq i \leq b-1} \{S_{(k)}^i\} \geq W_{(k)}^b$, we have

$$P\left(W_{(k)}^b \geq c\right) < P\left(\tilde{W}_{(k)}^b \geq c\right) = P\left(\max_{1 \leq i \leq b-1} \{S_{(k)}^i\} \geq c - n_k\right) \leq e^{-\theta_k(c - n_k)}$$

For all $2 \leq k \leq m$ and $1 \leq b \leq B$:

$$P\left(W_{(k)}^b \geq c\right) < e^{-\theta_k(c-n_k)},$$

where $\theta_k > 0$ is the solution to (20).

Union bound for safety guarantee. To ensure memory consumption does not exceed the bound at any batch $b \in \{1, \dots, B\}$ with probability at least $1 - \delta$, we apply a union bound across all segments and batches. For each segment $k \in \{2, \dots, m\}$ and batch index $b \in \{1, \dots, B\}$, we have:

$$P\left(W_{(k)}^b \geq c\right) < e^{-\theta_k(c-n_k)},$$

There are $m - 1$ segments (from $k = 2$ to m) and B batches (from $b = 1$ to B). We allocate the total failure probability δ across all segments and batches. The total number of events is $(m - 1) \cdot B$. Set the overflow probability for each segment and batch to:

$$P\left(W_{(k)}^b \geq c_k\right) \leq \frac{\delta}{(m - 1) \cdot B} \Rightarrow e^{-\theta_k(c_k - n_k)} \leq \frac{\delta}{(m - 1) \cdot B} \Rightarrow c_k \geq n_k + \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k}$$

If we choose

$$c_k = n_k + \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k},$$

then we have that

$$P\left(W_{(k)}^b \geq c_k\right) \leq \frac{\delta}{(m - 1) \cdot B}.$$

Next, we compute the joint probability of overflow across all segments and batches using a union bound:

$$P\left(\bigcup_{k=2}^m \bigcup_{b=1}^B \{W_{(k)}^b \geq c_k\}\right) \leq \sum_{k=2}^m \sum_{b=1}^B P\left(W_{(k)}^b \geq c_k\right) \leq (m - 1) \cdot B \cdot \frac{\delta}{(m - 1) \cdot B} = \delta$$

Thus:

$$P\left(W_{(k)}^b \leq c_k \text{ for all } \begin{matrix} k \in \{2, \dots, m\}, \\ b \in \{1, \dots, B\} \end{matrix}\right) \geq 1 - \delta.$$

So the memory consumption is bounded:

$$\text{Memory Consumption}^b = M^\pi + \sum_{k=2}^m W_{(k)}^b \cdot (l + l'_{k-1}) \leq M^\pi + \sum_{k=2}^m c_k \cdot (l + l'_{k-1})$$

where M^π is the peak batch memory usage as defined in Equation (13):

$$M^\pi = \sum_{j=1}^m n_j \left(l + \frac{L'_j}{2} \right) \Delta l'_j, \quad \text{with } L'_j = \sum_{i=1}^j l'_i, \Delta l'_j = l'_j - l'_{j-1}, l'_0 = 0.$$

Note that this differs from the M^π formula for the WAIT algorithm (Theorem 1), because Nested WAIT groups prompts into segments based on their unknown output lengths, leading to segment-wise memory accumulation

Substitute $c_k = n_k + \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k}$, we have that

$$\text{Memory Consumption}^b \leq M^\pi + \sum_{k=2}^m \left(n_k + \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k} \right) \cdot (l + l'_{k-1}).$$

Define the memory bound $\mathbf{M}^B$:

$$\mathbf{M}^B = M^\pi + \sum_{k=2}^m n_k \cdot (l + l'_{k-1}) + \sum_{k=2}^m \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k} \cdot (l + l'_{k-1}).$$

By the lower bound in Lemma 8, we can obtain the upper bound of the third term:

$$\sum_{k=2}^m \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k} \cdot (l + l'_{k-1}) \leq \sum_{k=2}^m \frac{n_{k-1}}{8(n_k - n_{k-1}p_k)} \ln\left(\frac{(m-1) \cdot B}{\delta}\right) \cdot (l + l'_{k-1})$$

And this $\mathbf{M}^B$ ensures that with probability at least $1 - \delta$, the memory consumption does not exceed $\mathbf{M}^B$ at any batch index $b \in \{1, \dots, B\}$.

Appendix D: Proof of Theorem 3

Proof Outline. This proof extends Nested WAIT to time-varying arrival rates $\lambda_j(t)$. Time-varying rates make exact load balance impossible, since the optimal threshold depends on the instantaneous arrival pattern. The *worst-case domination strategy* maintains load balance even at peak rates: by designing thresholds to handle worst-case arrivals, we ensure system stability across all scenarios. This conservative approach prevents eviction throughout the time horizon. The structure proceeds as follows:

1. **First segment analysis:** Poisson arrivals with time-varying rates are dominated by a time-invariant process using worst-case aggregate rate $\Lambda^\pi = \sup_b \{ \sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}] \}$. We couple with a Lindley process and apply Kingman's inequality.

2. **Subsequent segments analysis:** Binomial arrivals with time-varying thinning probabilities $p_k(b)$ are dominated by worst-case $p_k^* = \sup_b \{p_k(b)\}$. This ensures safety across all batch compositions. We apply Lindley coupling to each segment.

3. **Throughput and high-probability bounds:** Asymptotic optimality follows from the time-invariant dominated processes. High-probability memory bounds follow the same martingale construction, using worst-case parameters to prevent eviction.

We use b to denote the *batch index* and $k \in \{1, \dots, m\}$ to denote the *segment index*. The total number of batches is denoted by B .

D.1. First Segment Analysis

For the first segment ($k = 1$), the state transition rule is

$$W_{(1)}^{b+1} = W_{(1)}^b + Y_{(1)}^b - n_1 \cdot \mathbf{1}\{W_{(1)}^b + Y_{(1)}^b \geq n_1\}, \quad Y_{(1)}^b \sim \text{Poisson}\left(\sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_b]\right),$$

where $t(b)$ denotes the start time of batch b and ΔT_b denotes the processing time of batch b .

Worst-case domination. Time-varying arrival rates prevent exact load balance since optimal thresholds depend on instantaneous arrival patterns. To maintain stability across all scenarios, we use worst-case domination: design thresholds to handle peak arrival rates, ensuring eviction prevention even under worst-case conditions. Since $\Delta T_b \leq \Delta T_{[1, \dots, m]}$ (the time for a full batch containing all m types), we construct a dominating process $\{\bar{W}_{(1)}^b\}$:

$$\bar{W}_{(1)}^{b+1} = \bar{W}_{(1)}^b + \bar{Y}_{(1)}^b - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^b + \bar{Y}_{(1)}^b \geq n_1\}, \quad \bar{Y}_{(1)}^b \sim \text{Poisson}\left(\sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}]\right).$$

By (15), the aggregate arrival rate during any batch is bounded by the worst-case aggregate rate:

$$\sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}] \leq \sup_{\forall b} \left\{ \sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}] \right\} < n_1.$$

Define $\Lambda^\pi = \sup_{\forall b} \left\{ \sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}] \right\} < n_1$ as the worst-case aggregate arrival rate.

This enables a time-invariant dominating process that provides tractable bounds:

$$\tilde{W}_{(1)}^{b+1} = \tilde{W}_{(1)}^b + \tilde{Y}_{(1)}^b - n_1 \cdot \mathbf{1}\{\tilde{W}_{(1)}^b + \tilde{Y}_{(1)}^b \geq n_1\}, \quad \tilde{Y}_{(1)}^b \sim \text{Poisson}(\Lambda^\pi).$$

This time-invariant process enables classical queueing analysis. Define $X_{(1)}^b = \tilde{Y}_{(1)}^b - n_1$. The process $\{\tilde{W}_{(1)}^b\}$ is dominated by a Lindley process:

$$\bar{W}_{(1)}^0 = 2n_1, \quad \bar{W}_{(1)}^{b+1} = \max\{2n_1, \bar{W}_{(1)}^b + X_{(1)}^b\}, \quad \forall b \in \mathbb{N}.$$

Thus we have the dominance chain:

$$W_{(1)}^b \leq \bar{W}_{(1)}^b \leq \tilde{W}_{(1)}^b \leq \overline{W}_{(1)}^b, \quad \forall b \in \mathbb{N}.$$

Since $n_1 > \Lambda^\pi$, the process $\{\bar{W}_{(1)}^b\}$ has negative drift. By Kingman's inequality (Kingman 1962), with

$$\text{Var}(X_{(1)}^b) = \Lambda^\pi, \quad \left| \mathbb{E} \left[X_{(1)}^b \right] \right| = n_1 - \Lambda^\pi,$$

we obtain

$$\mathbb{E} \left[W_{(1)}^b \right] \leq 2n_1 + \frac{\Lambda^\pi}{2(n_1 - \Lambda^\pi)}.$$

D.2. Subsequent Segments Analysis ($k \geq 2$)

For segment k ($2 \leq k \leq m$), arrivals come from prompts that completed segment $k - 1$. The batch index b for segment k counts only batches containing segment $k - 1$. These indices differ across segments but are bounded by the total batch count B .

Time-varying thinning probabilities. Consider the b -th arrival to segment k , which comes from the b -th batch containing segment $k - 1$. The thinning probability $p_k(b)$ depends on the prompt composition in this batch and varies over time. Although exact arrival times are unknown, arrivals occur within some interval $[t, t + \Delta t]$ where $t + \Delta t \leq t(b, k - 1)$, the starting time of the b -th batch containing segment $k - 1$. To handle this time variation, we use the worst-case thinning probability. By (15):

$$p_k(b) = \frac{\sum_{j=k+1}^m \lambda_j[t, t + \Delta t]}{\sum_{j=k}^m \lambda_j[t, t + \Delta t]} \leq p_k^* < \frac{n_k}{n_{k-1}}, \quad \forall b \in \mathbb{N}.$$

The arrival process is

$$Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k(b)), \quad \forall b \geq 0,$$

and the state transition rule is

$$W_{(k)}^{b+1} = W_{(k)}^b + Y_{(k)}^b - n_k \cdot \mathbf{1}\{W_{(k)}^b + Y_{(k)}^b \geq n_k\},$$

where b indexes batches containing segment $k - 1$, since new arrivals to segment k occur only when segment $k - 1$ is processed.

Define $X_{(k)}^b = Y_{(k)}^b - n_k$. Although the distribution $Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k(b))$ is time-varying, the coupling construction follows Lemma 7:

LEMMA 9 (Coupling for Time-Varying Case). *Define a coupled process $\{\tilde{W}_{(k)}^b\}$ by:*

$$\begin{aligned}\tilde{W}_{(k)}^0 &= n_k, \\ \tilde{W}_{(k)}^{b+1} &= \max\{n_k, \tilde{W}_{(k)}^b + X_{(k)}^b\}, \\ X_{(k)}^b &= Y_{(k)}^b - n_k.\end{aligned}$$

Then $\tilde{W}_{(k)}^b \geq W_{(k)}^b$ for all $b \in \mathbb{N}$.

To apply Kingman's inequality (Kingman 1962), we construct a time-invariant dominating process:

$$\begin{aligned}\bar{W}_{(k)}^0 &= n_k, \\ \bar{W}_{(k)}^{b+1} &= \max\{n_k, \bar{W}_{(k)}^b + \bar{X}_{(k)}^b\}, \\ \bar{X}_{(k)}^b &= \bar{Y}_{(k)}^b - n_k, \\ \bar{Y}_{(k)}^b &\sim \text{Binomial}(n_{k-1}, p_k^*), \quad \forall b \in \mathbb{N}.\end{aligned}$$

Then we have the dominance chain:

$$\bar{W}_{(k)}^b \geq \tilde{W}_{(k)}^b \geq W_{(k)}^b, \quad \forall b \in \mathbb{N}.$$

The process $\bar{W}_{(k)}^b$ has the Lindley representation:

$$\begin{aligned}\bar{W}_{(k)}^0 &= n_k, \\ \bar{W}_{(k)}^{b+1} &= n_k + \max_{0 \leq i \leq b} \{\bar{S}_{(k)}^i\}, \\ \text{where } \bar{S}_{(k)}^0 &= 0, \bar{S}_{(k)}^i = \sum_{r=1}^i \bar{X}_{(k)}^r, \quad 1 \leq i \leq b.\end{aligned}$$

Since

$$\mathbb{E} \left[\bar{X}_{(k)}^b \right] = n_{k-1} \cdot p_k^* - n_k < 0, \quad \forall b \in \mathbb{N},$$

the coupled process $\{\bar{W}_{(k)}^b\}$ has negative drift. With

$$\text{Var}(\bar{X}_{(k)}^b) = n_{k-1} p_k^* (1 - p_k^*), \quad \left| \mathbb{E} \left[\bar{X}_{(k)}^b \right] \right| = n_k - p_k^* n_{k-1},$$

Kingman's inequality (Kingman 1962) yields:

$$\mathbb{E} \left[\max_{0 \leq i \leq b} \{\bar{S}_{(k)}^i\} \right] \leq \frac{n_{k-1} p_k^* (1 - p_k^*)}{2(n_k - n_{k-1} p_k^*)}.$$

Thus, the expected queue length is bounded:

$$\mathbb{E} \left[W_{(k)}^b \right] \leq \mathbb{E} \left[\tilde{W}_{(k)}^b \right] \leq \mathbb{E} \left[\bar{W}_{(k)}^b \right] \leq n_k + \frac{n_{k-1} p_k^* (1 - p_k^*)}{2(n_k - n_{k-1} p_k^*)}.$$

D.3. Throughput and High-Probability Bounds

The discussion of asymptotic optimal throughput under bounded latency and TTFT follows Appendix C. For high-probability bounds, the analysis follows the same martingale construction applied to the time-invariant coupled process $\{\bar{W}_{(k)}^b\}$ defined above. By the same argument as in Appendix C, with $\bar{X}_{(k)}^r = \bar{Y}_{(k)}^r - n_k$, $\bar{Y}_{(k)}^r \sim \text{Binomial}(n_{k-1}, p_k^*)$, and $p_k^* < n_k/n_{k-1}$, the memory bound $\mathbf{M}^B$ is:

$$\mathbf{M}^B = M^\pi + \sum_{k=2}^m n_k \cdot (l + l'_{k-1}) + \sum_{k=2}^m \theta_k^{-1} \ln \left(\frac{(m-1) \cdot B}{\delta} \right) \cdot (l + l'_{k-1}),$$

where θ_k ($2 \leq k \leq m$) is the unique positive solution to:

$$e^{-\theta_k n_k} (1 - p_k^* + p_k^* e^{\theta_k})^{n_{k-1}} = 1. \quad (21)$$

Since $p_k^* = \sup_b \{p_k(b)\}$ represents the worst-case thinning probability, by the strictly increasing property of θ_k in Lemma 8, this yields the smallest θ_k and hence the largest memory bound term. The lower bound for θ_k follows from the same analysis in Appendix C.

Appendix E: Proof of Lemmas

Throughout this section, we use b for the batch index and B for the total number of batches.

E.1. Proof of Lemma 1

To prove this lemma, we take expectations on both sides of the state transition equation:

$$\mathbb{E} [W^{b+1}] = \mathbb{E} [W^b] + \lambda - \lambda \mathbb{P}(W^b + X^b \geq \lambda).$$

Summing from $b = 0$ to $B - 1$, and noting that $W^0 = 0$, we obtain:

$$\mathbb{E} [W^B] = \lambda B - \lambda \mathbb{E} [B - B_{\text{stuck}}] = \lambda \cdot \mathbb{E} [B_{\text{stuck}}].$$

E.2. Proof of Lemma 2

We prove by induction on the batch index b . The coupled process $\{\tilde{W}^b\}$ starts with a safety buffer of 2λ (compared to the real process starting at 0), ensuring it remains above $W^b + \lambda$ throughout. For the inductive step, we consider two cases based on whether the system can process a batch or experiences a stuck iteration.

Base case. For $b = 0$: $\tilde{W}^0 = 2\lambda \geq \lambda = W^0 + \lambda$.

Inductive step. Assume $\tilde{W}^b \geq W^b + \lambda$ for some $b \geq 0$. We show $\tilde{W}^{b+1} \geq W^{b+1} + \lambda$ by considering two cases:

Case 1 (Batch processed): If $W^b + X^b \geq \lambda$, the queue has accumulated enough prompts to process a batch, so $W^{b+1} = W^b + X^b - \lambda$. Then

$$\tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\} \geq \tilde{W}^b + X^b - \lambda \geq (W^b + \lambda) + X^b - \lambda = W^{b+1} + \lambda.$$

Case 2 (Stuck iteration): If $W^b + X^b < \lambda$, insufficient prompts are available and the system cannot process a batch (stuck iteration). The queue simply accumulates arrivals: $W^{b+1} = W^b + X^b$. The safety buffer ensures dominance:

$$\tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\} \geq 2\lambda = \lambda + \lambda > W^{b+1} + \lambda.$$

By induction, $\tilde{W}^b \geq W^b + \lambda$ for all $b \in \mathbb{N}$.

E.3. Proof of Lemma 6

Proof idea. The coupled process dominates the real process by maintaining a consistent full-batch processing schedule, while the real process may skip type j in some batches when $W_{(j)}^b < n_j$. We analyze periods when type j actively joins batches and show the coupled process makes at most one additional batch iteration, preserving dominance.

Proof. Since $\tilde{W}_{(j)}^b \geq 2n_j$, when $W_{(j)}^b < n_j$, we trivially have $\tilde{W}_{(j)}^b \geq W_{(j)}^b$.

Initially, $W_{(j)}^0 = 0$. As arrivals accumulate, $W_{(j)}^b$ reaches n_j and type j joins the batch. After processing, $W_{(j)}^b$ may drop below n_j , and the cycle repeats.

Consider a period $[b_0, b_1]$ when $W_{(j)}^b \geq n_j$, meaning type j is actively joining batches in the real process. Let $\tau = \max\{b \leq b_0 : W_{(j)}^b = n_j\}$ be the last time before b_0 when the queue length equals exactly n_j . We compare the number of batch iterations between the real and coupled processes over the interval $[\tau, b]$.

For any $b \in [b_0, b_1]$, the elapsed time is $t(b) - t(\tau)$, where $t(b)$ denotes the time at batch b . Since each batch takes at most $\Delta T_{[1, \dots, m]}$ time (the full-batch processing time), we can bound the number of batch iterations $I_{\tau \rightarrow b}$ (real process) and $\tilde{I}_{\tau \rightarrow b}$ (coupled process):

$$I_{\tau \rightarrow b} \geq \lfloor \frac{t(b) - t(\tau)}{\Delta T_{[1, \dots, m]}} \rfloor, \quad \tilde{I}_{\tau \rightarrow b} \leq 1 + \lfloor \frac{t(b) - t(\tau)}{\Delta T_{[1, \dots, m]}} \rfloor \Rightarrow \tilde{I}_{\tau \rightarrow b} \leq 1 + I_{\tau \rightarrow b}.$$

Thus, the coupled process makes at most one more iteration than the real process after τ .

At time τ , the real process may be in a batch without type j (since not all batches contain all types), while the coupled process just completed a full batch containing all types. This yields the initial dominance $\tilde{W}_{(j)}^\tau \geq n_j = W_{(j)}^\tau$.

For batches in the interval $(\tau, b]$, the arrival dynamics are identical for both processes. Since type j joins every batch in the real process during this period (as $W_{(j)}^b \geq n_j$), but the coupled process completes at most one additional batch, the dominance is maintained. Thus $\tilde{W}_{(j)}^b \geq W_{(j)}^b$ for all $b \geq 0$.

E.4. Proof of Lemma 7

The proof follows the same induction structure as Lemma 2, adapted to the Nested WAIT setting where arrivals follow binomial thinning from segment $k - 1$ completions.

Base case. For $b = 0$: $\tilde{W}_{(k)}^0 = n_k \geq 0 = W_{(k)}^0$.

Inductive step. Assume $\tilde{W}_{(k)}^b \geq W_{(k)}^b$ for some $b \geq 0$. We show $\tilde{W}_{(k)}^{b+1} \geq W_{(k)}^{b+1}$ by case analysis:

Case 1 (Segment batch processed): If $W_{(k)}^b + Y_{(k)}^b \geq n_k$, then $W_{(k)}^{b+1} = W_{(k)}^b + Y_{(k)}^b - n_k$, so

$$\tilde{W}_{(k)}^{b+1} = \max\{n_k, \tilde{W}_{(k)}^b + X_{(k)}^b\} \geq \tilde{W}_{(k)}^b + X_{(k)}^b \geq W_{(k)}^b + Y_{(k)}^b - n_k = W_{(k)}^{b+1}.$$

Case 2 (Segment stuck): If $W_{(k)}^b + Y_{(k)}^b < n_k$, then $W_{(k)}^{b+1} = W_{(k)}^b + Y_{(k)}^b$, so

$$\tilde{W}_{(k)}^{b+1} = \max\{n_k, \tilde{W}_{(k)}^b + X_{(k)}^b\} \geq n_k > W_{(k)}^{b+1}.$$

By induction, $\tilde{W}_{(k)}^b \geq W_{(k)}^b$ for all $b \in \mathbb{N}$.

E.5. Proof of Lemma 8

This lemma establishes existence and properties of the parameter $\theta_k > 0$ that makes the exponential process $e^{\theta_k S_{(k)}^i}$ a martingale. This enables Doob's inequality to convert expected queue length bounds into high-probability bounds, preventing eviction cascades.

Existence and uniqueness. For simplicity, we denote $\theta = \theta_k$. Define the function

$$f(\theta) = e^{-\theta n_k} (1 - p_k + p_k e^\theta)^{n_{k-1}}.$$

We seek $\theta > 0$ such that $f(\theta) = 1$. First, evaluate $f(\theta)$ at $\theta = 0$:

$$f(0) = e^0 (1 - p_k + p_k \cdot 1)^{n_{k-1}} = 1.$$

Thus, $\theta = 0$ is a trivial solution, but we need $\theta > 0$ to obtain useful high-probability bounds. To analyze $f(\theta)$ for $\theta > 0$, we consider the logarithm

$$g(\theta) = \ln f(\theta) = -\theta n_k + n_{k-1} \ln(1 - p_k + p_k e^\theta),$$

$$g'(\theta) = -n_k + n_{k-1} \cdot \frac{p_k e^\theta}{1 - p_k + p_k e^\theta}.$$

At $\theta = 0$,

$$g(0) = 0, \quad g'(0) = -n_k + n_{k-1} p_k.$$

Since $\mathbb{E}[X_{(k)}^b] = n_{k-1} p_k - n_k < 0$, we have $g'(0) < 0$, so $f(\theta)$ is decreasing near $\theta = 0$. Consider the second derivative:

$$g''(\theta) = n_{k-1} \cdot \frac{p_k e^\theta (1 - p_k)}{(1 - p_k + p_k e^\theta)^2} > 0 \quad \text{for } \theta > 0,$$

since $p_k \in (0, 1)$ and $n_{k-1} > 0$. Thus, $g(\theta)$ is strictly convex. Convexity ensures that $g(\theta)$ can cross zero at most once for $\theta > 0$, guaranteeing uniqueness. Now, consider the limit as $\theta \rightarrow \infty$:

$$f(\theta) \approx p_k^{n_{k-1}} e^{\theta(n_{k-1} - n_k)}.$$

By our settings, $n_{k-1} > n_k$, which ensures $f(\theta) \rightarrow \infty$ as $\theta \rightarrow \infty$. Thus, $f(\theta)$ is continuous, convex, starts at $f(0) = 1$, decreases below 1 for small $\theta > 0$ (since $g'(0) = n_{k-1} p_k - n_k < 0$ by negative drift), and increases to infinity. By the intermediate value theorem and convexity, there exists a unique $\theta > 0$ such that $f(\theta) = 1$.

Monotonicity in drift. Next, we prove that the solution $\theta > 0$ is strictly increasing with respect to the drift $D = n_k - n_{k-1} p_k$. This shows that tighter capacity (larger D , more negative drift) leads to larger θ and thus tighter high-probability bounds. Recall that $g(0) = 0$, $g'(0) = -D < 0$, and $g''(\theta) > 0$ for $\theta > 0$, so $g(\theta)$ is strictly convex. The unique positive solution $\theta > 0$ occurs where $g(\theta)$ crosses zero after initially decreasing from $g(0) = 0$.

Consider two values D_1 and D_2 such that $D_1 < D_2$, with corresponding functions $g_1(\theta) = -\theta(n_{k-1} p_k + D_1) + n_{k-1} \ln(1 - p_k + p_k e^\theta)$ and $g_2(\theta) = -\theta(n_{k-1} p_k + D_2) + n_{k-1} \ln(1 - p_k + p_k e^\theta)$, and solutions θ_1 and θ_2 , respectively. At $\theta = 0$, we have $g'_1(0) = -D_1 > g'_2(0) = -D_2$, meaning $g_2(\theta)$ decreases more steeply from zero than $g_1(\theta)$.

For a fixed $\theta > 0$, the partial derivative $\frac{\partial g}{\partial D} = -\theta < 0$, so increasing D decreases $g(\theta)$ pointwise. Since $g(\theta)$ is convex and approaches infinity as $\theta \rightarrow \infty$, the zero crossing of $g(\theta; D)$ shifts to a larger θ when D increases. Hence, if $D_2 > D_1$, then $\theta_2 > \theta_1$. Operationally, this means larger safety margins yield tighter high-probability bounds.

Drift-to- θ scaling. Finally, we characterize how the solution $\theta > 0$ scales with drift D , n_{k-1} , and p_k . When the drift $D = n_k - n_{k-1}p_k$ is small (system near critical load), the solution θ is also small. To derive an explicit approximation, we expand $g(\theta)$ around $\theta = 0$ using Taylor's theorem:

$$g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(0)\theta^2 + O(\theta^3) = 0 - D\theta + \frac{1}{2}n_{k-1}p_k(1-p_k)\theta^2 + O(\theta^3).$$

Setting $g(\theta) = 0$ and neglecting higher-order terms, we obtain the quadratic approximation

$$-D\theta + \frac{1}{2}n_{k-1}p_k(1-p_k)\theta^2 \approx 0 \implies \theta \approx \frac{2D}{n_{k-1}p_k(1-p_k)}.$$

This shows that for small D , the solution scales linearly: $\theta = O(D)$. Specifically, $\theta \approx \frac{2(n_k - n_{k-1}p_k)}{n_{k-1}p_k(1-p_k)}$. This linear scaling implies that the high-probability memory bound grows logarithmically in B/δ , with the coefficient inversely proportional to the drift D .

General lower bound. To obtain a uniform lower bound valid for all $D > 0$ (not just small D), we apply Taylor's theorem with remainder:

$$g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(c)\theta^2 = -D\theta + \frac{1}{2}g''(c)\theta^2 = 0 \implies \theta = \frac{2D}{g''(c)}. \quad \text{for some } c \in (0, \theta)$$

To find an upper bound for $g''(c)$, we write $g''(x) = n_{k-1} \cdot \frac{p_k e^x (1-p_k)}{(1-p_k + p_k e^x)^2}$ and define $h(y) = \frac{p_k y (1-p_k)}{(1-p_k + p_k y)^2}$ where $y = e^x \geq 1$. By differentiation, $h(y)$ achieves a maximum of $\frac{1}{4}$ over $y \geq 1$ and $0 < p_k < 1$ (attained when the numerator and denominator are balanced). Thus we have that $g''(x) \leq n_{k-1} \cdot \frac{1}{4} = \frac{n_{k-1}}{4} \implies \theta = \frac{2D}{g''(c)} \geq \frac{2D}{n_{k-1}/4} = \frac{8D}{n_{k-1}}$. So a general lower bound is:

$$\theta \geq \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}. \quad (22)$$